Árboles. Árboles binarios y árboles ordenados

Algoritmos y Estructuras de Datos II
Ingeniería en Sistemas

Introducción

Un árbol es una **estructura de datos no lineal**, lo que significa que los datos no se organizan en forma secuencial como en un array o lista.

En lugar de eso, los elementos se organizan de forma **jerárquica**, similar a un árbol genealógico.

Esta compuesta por nodos conectados por aristas. Se utiliza para representar relaciones entre elementos, donde cada nodo contiene datos y está conectado a otros nodos.

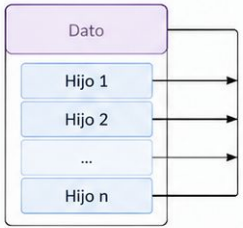
¿QUÉ ESTRUCTURA DE DATOS SOPORTA A UN ÁRBOL?

La base de un árbol son nodos enlazados entre sí mediante referencias (punteroo).

Un árbol **NO** es una estructura "mágica". Internamente se construye con nodos que guardan un dato y referencias a sus hijos.

1. IDEA BASE DE UN ÁRBOL

ESTRUCTURA DE UN NODO



Cada nodo guarda:

- un dato
- referencias a sus hijos

En una lista cada nodo apunta normalmente a 1 siguiente.
En un árbol un nodo puede apuntar a varios hijos.

2. COMPARACIÓN RÁPIDA

LISTA ENLAZADA



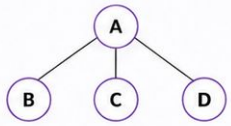
Cada nodo tiene:

- dato
- referencia al siguiente

```
struct NodoLista {
  int dato;
  NodoLista* siguiente;
};
```

VS

ÁRBOL



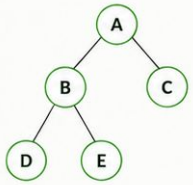
Cada nodo tiene:

- dato
- referencias a varios hijos

```
struct NodoArbol {
  int dato;
  // referencias a hijos
};
```

3. DEPENDE DEL TIPO DE ÁRBOL

ÁRBOL BINARIO (cada nodo tiene hasta 2 hijos)

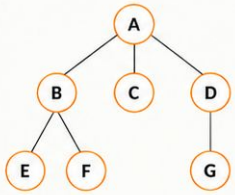


Cada nodo tiene:

- referencia al hijo izquierdo
- referencia al hijo derecho

```
struct NodoBinario {
  int dato;
  NodoBinario* izq;
  NodoBinario* der;
};
```

ÁRBOL GENERAL (cada nodo puede tener muchos hijos)



Cada nodo puede tener muchos hijos.

¿Cómo se almacenan los hijos?

- Lista de hijos (lista enlazada)
- Vector/Arreglo dinámico
- Otra estructura de colección

```
struct NodoGeneral {
  int dato;
  Lista<NodoGeneral*> hijos;
  // o: vector<NodoGeneral*>
};
```

4. RESUMEN VISUAL

¿QUÉ SOPORTA A UN ÁRBOL?

ÁRBOL

NODOS ENLAZADOS (PUNTEROS / REFERENCIAS)

Los nodos están conectados entre sí mediante punteros o referencias.

CADA NODO GUARDA:

- dato
- referencias a sus hijos

LAS REFERENCIAS FORMAN LA ESTRUCTURA JERÁRQUICA

Estas referencias definen la relación padre–hijo entre los nodos.

IDEA CLAVE

Un árbol se implementa mediante nodos conectados por referencias.

Conceptualmente deriva de la idea de listas enlazadas, pero permitiendo múltiples conexiones por nodo.

EN POCAS PALABRAS

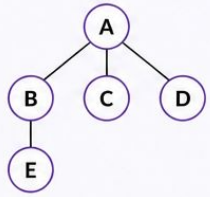


La base de un árbol son nodos enlazados mediante punteros o referencias.

PARA TENER EN CUENTA

- En todos los casos, lo importante es que las referencias preserven la relación padre–hijo, manteniendo la estructura jerárquica del árbol.
- Según el tipo de árbol, cada nodo puede tener 1, 2 o muchos hijos.
- La forma en que se almacenan y recorren estas referencias define las operaciones del árbol (búsquedas, inserciones, eliminaciones, recorridos, etc.).

EJEMPLO RÁPIDO



Cada nodo apunta a sus hijos:

- A → B, C, D
- B → E
- C → (sin hijos)
- D → (sin hijos)
- E → (sin hijos)

EN RESUMEN: Un árbol se soporta con una estructura de nodos enlazados por referencias, donde cada nodo conoce a sus hijos.
¡Eso es lo que le da su forma y funcionalidad!

Concepto General

Un árbol está formado por:

- **Nodos** → contienen datos
- **Ramas** → conectan los nodos

Tiene una estructura jerárquica que comienza desde un nodo principal llamado:
raíz.

Los árboles se utilizan porque permiten:

- Representar información compleja de forma organizada
- Realizar búsquedas de manera eficiente
- Modelar estructuras reales (carpetas, decisiones, expresiones)

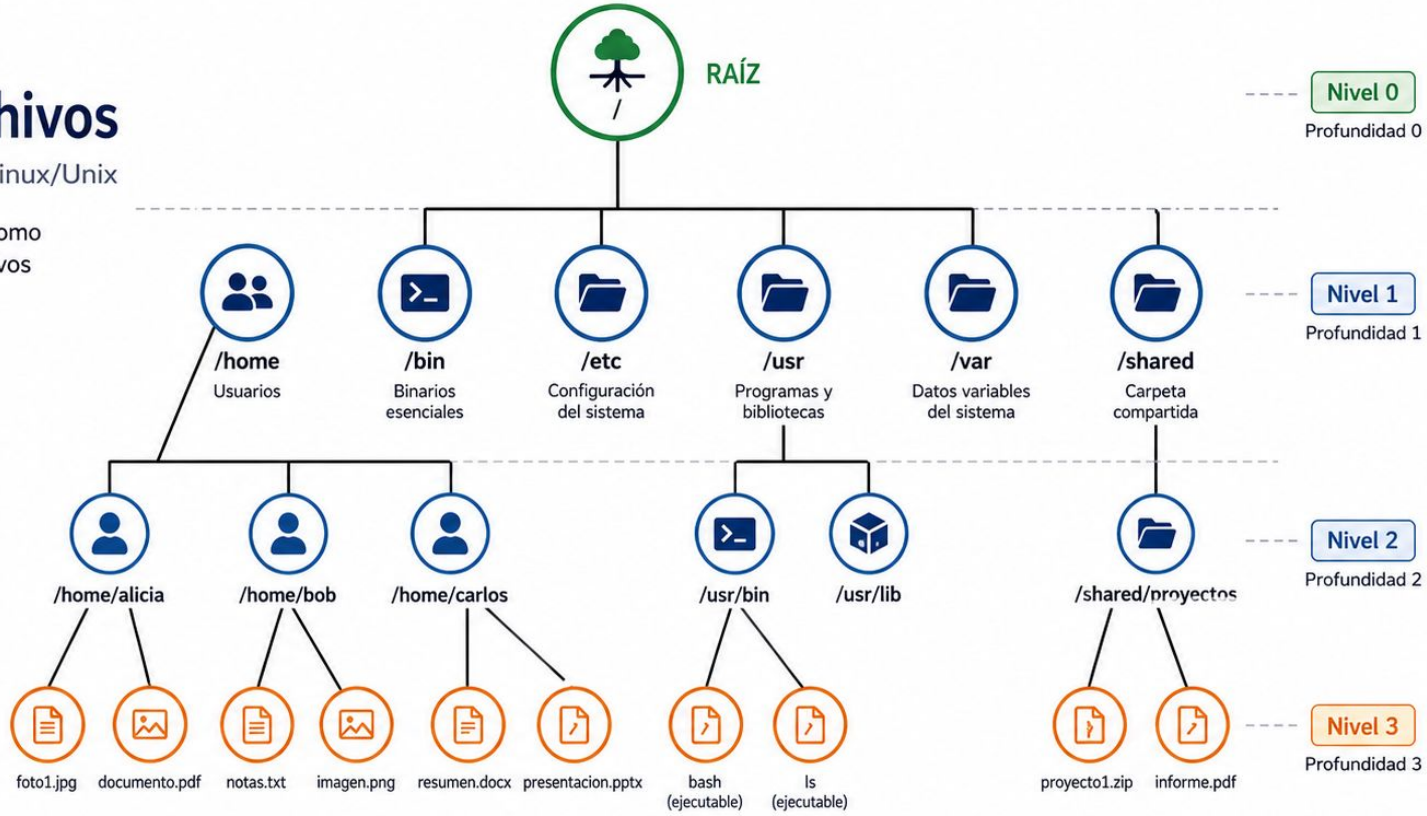
Ejemplo: sistema de archivos de un sistema operativo.

Jerarquía del Sistema de Archivos

Modelo de árbol en sistemas Linux/Unix

El sistema de archives se organiza como una jerarquía de directorios y archivos que parte de una raíz común.

- **Jerarquía de 4 niveles (0 a 3)**
  Nivel 0 = raíz / Nivel 3 = archivos
- **Profundidad: 0 a 3**
  La profundidad indica la distancia desde la raíz.
- **Nodos internos = Directorios**
  Contienen subdirectorios o archivos.
- **Nodos hoja = Archivos**
  No contienen elementos hijos.
- **Ramas (aristas) = Conexiones**
  Representan las relaciones (rutas) entre directorios y archivos.



- **Nodo interno (directorio)**
  Puede contener otros nodos.
- **Nodo hoja (archivo)**
  No contiene otros nodos.

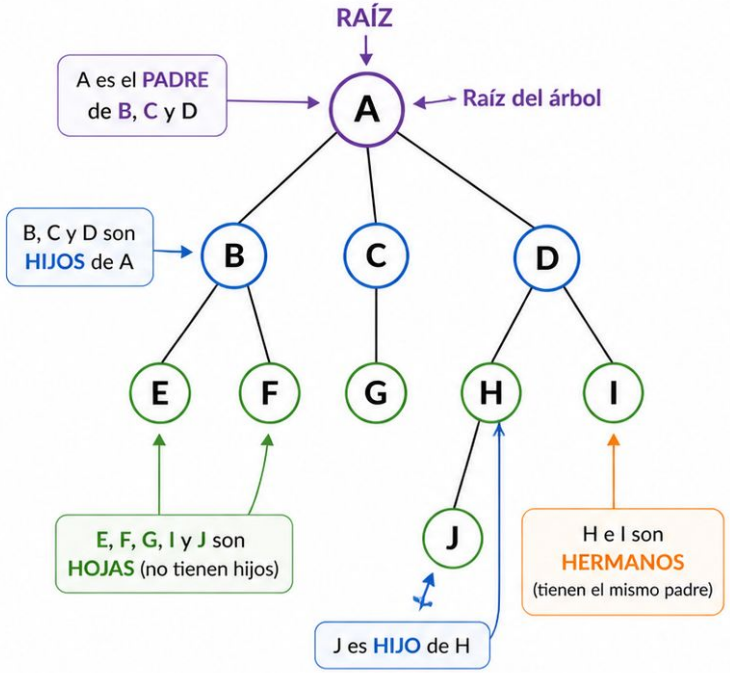
Este árbol representa cómo se organiza la información en un sistema de archivos.
Cada ruta única desde la raíz hasta un nodo define la ubicación de un archivo o directorio.

Ejemplo de ruta:
/home/alicia/documento.pdf



CONCEPTOS BÁSICOS EN UN ÁRBOL

Un **árbol** es una estructura jerárquica de **nodos** conectados.
Cada nodo puede tener **cero o más hijos.**



PADRE
Nodo que tiene uno o más hijos.

Ejemplo: A es padre de B, C y D.

HIJO
Nodo que depende de otro nodo (superior).

Ejemplo: B, C y D son hijos de A.
J es hijo de H.

HERMANO
Nodos que tienen el mismo padre.

Ejemplo: H e I son hermanos.
E y F son hermanos.

HOJA
Nodo que no tiene hijos.

Ejemplo: E, F, G, I y J son hojas.

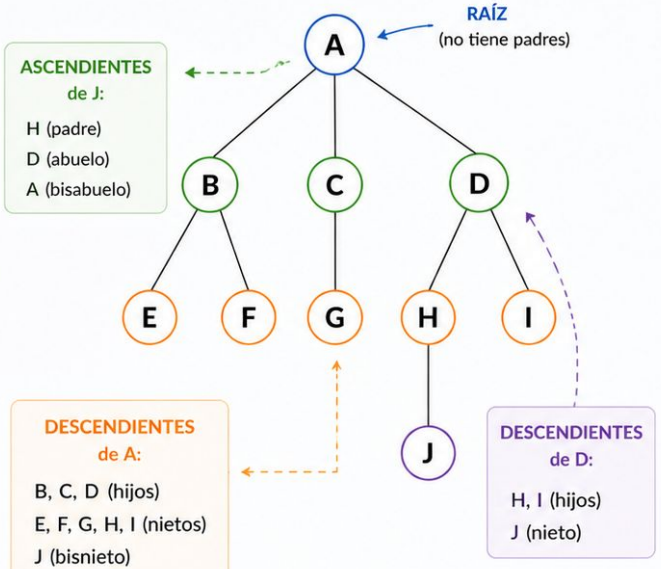
RESUMEN:

- **PADRE:** tiene hijos.
- **HIJO:** depende de otro nodo.
- **HERMANO:** comparten el mismo padre.
- **HOJA:** no tiene hijos.

RELACIÓN ENTRE NODOS EN UN ÁRBOL

En un **árbol**, los nodos se relacionan entre sí por su **posición jerárquica**.
Estas relaciones nos ayudan a entender la estructura y a recorrer el árbol.

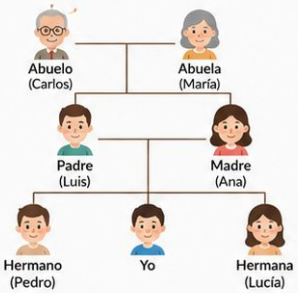
EJEMPLO DE ÁRBOL



CONCEPTOS CLAVE

ASCENDIENTES	Son los nodos que están por encima de un nodo: padres, abuelos, bisabuelos, etc.
DESCENDIENTES	Son los nodos que están por debajo de un nodo: hijos, nietos, bisnietos, etc.
NODO INTERNO	Es un nodo que tiene uno o más hijos. (Ejemplo: A, B, C, D, H)
NODO HOJA	Es un nodo que no tiene hijos. (Ejemplo: E, F, G, I, J)

SIMILITUD CON UN ÁRBOL GENEALÓGICO



- **ASCENDIENTES** de "Yo": Padre, Abuela, Abuelo
- **DESCENDIENTES** de Abuela (María): Padre, Tíos, Primos, etc.
- **NODO INTERNO:** Abuela, Padre, Madre
- **NODO HOJA:** Pedro, Yo, Lucía

EN RESUMEN

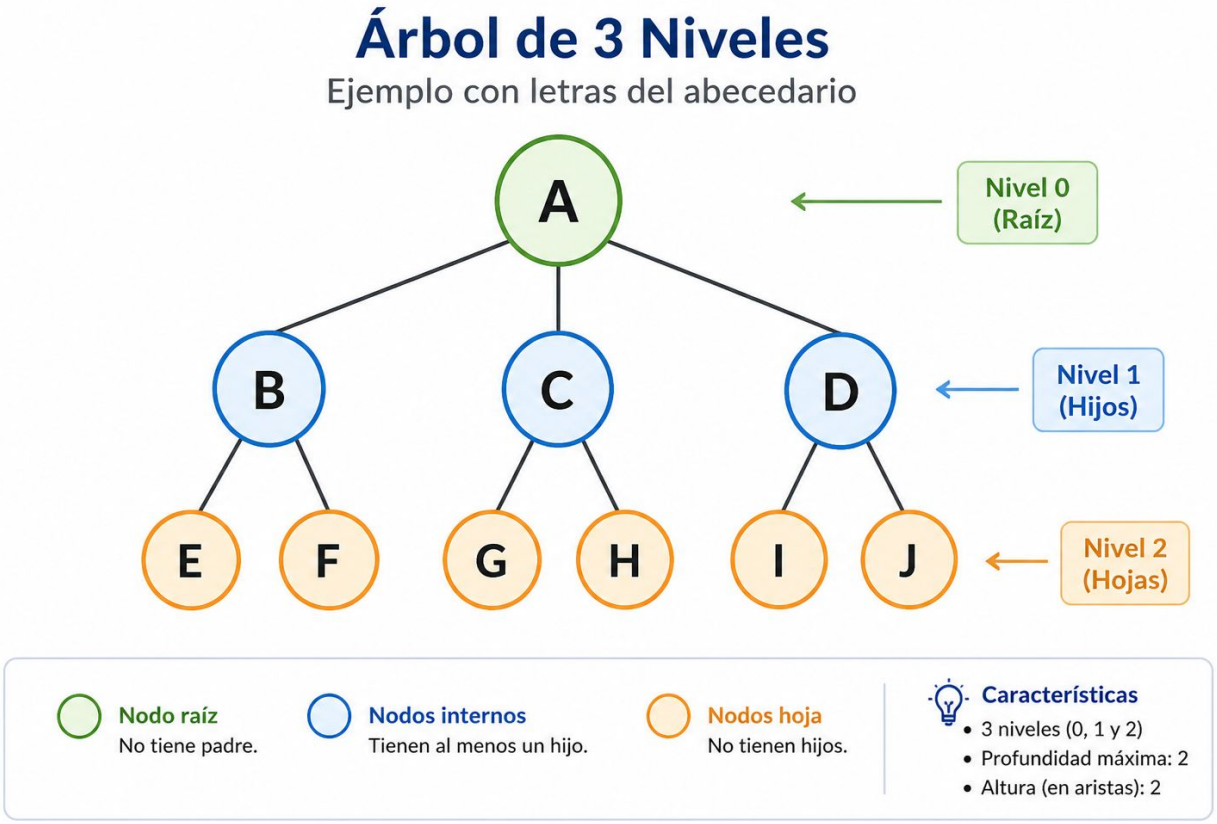
- **Ascendientes:** nodos que están por encima (padres, abuelos, etc.).
- **Descendientes:** nodos que están por debajo (hijos, nietos, etc.).
- **Nodo interno:** tiene uno o más hijos.
- **Nodo hoja:** no tiene hijos.
- Estas relaciones forman la estructura jerárquica de un árbol, al igual que en un árbol genealógico.

Nivel de un Nodo

El nivel indica qué tan lejos

está un nodo de la raíz:

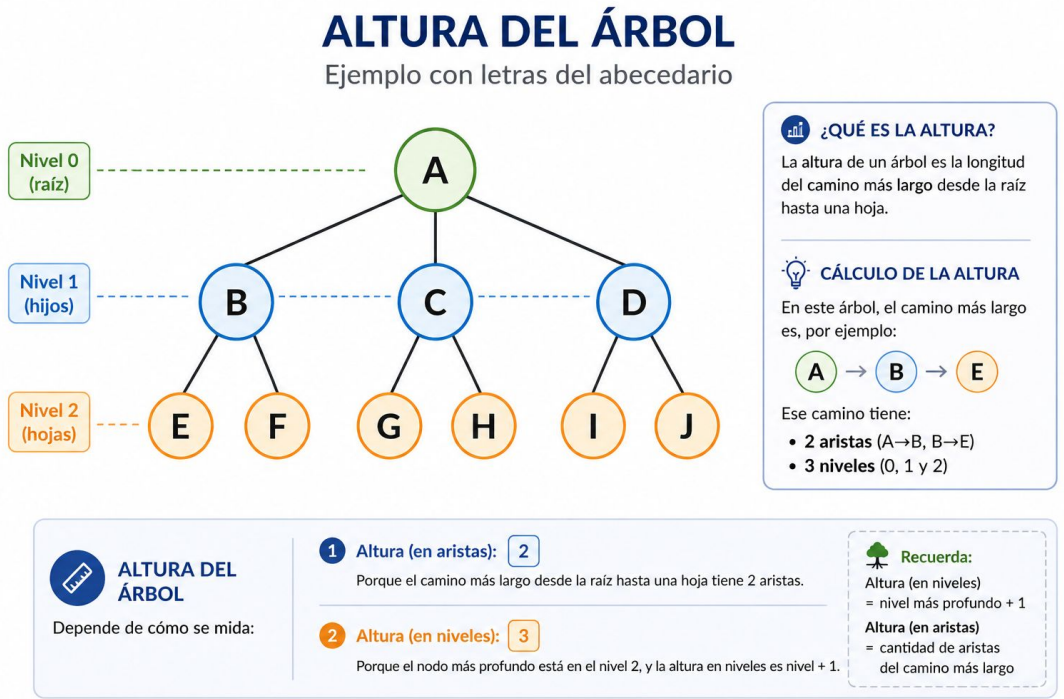
- Raíz → nivel 0
- Hijos → nivel 1
- Nietos → nivel 2



Altura del Árbol

La altura es:
El nivel del nodo más profundo + 1

También se puede ver como:
El camino más largo desde la raíz hasta una hoja

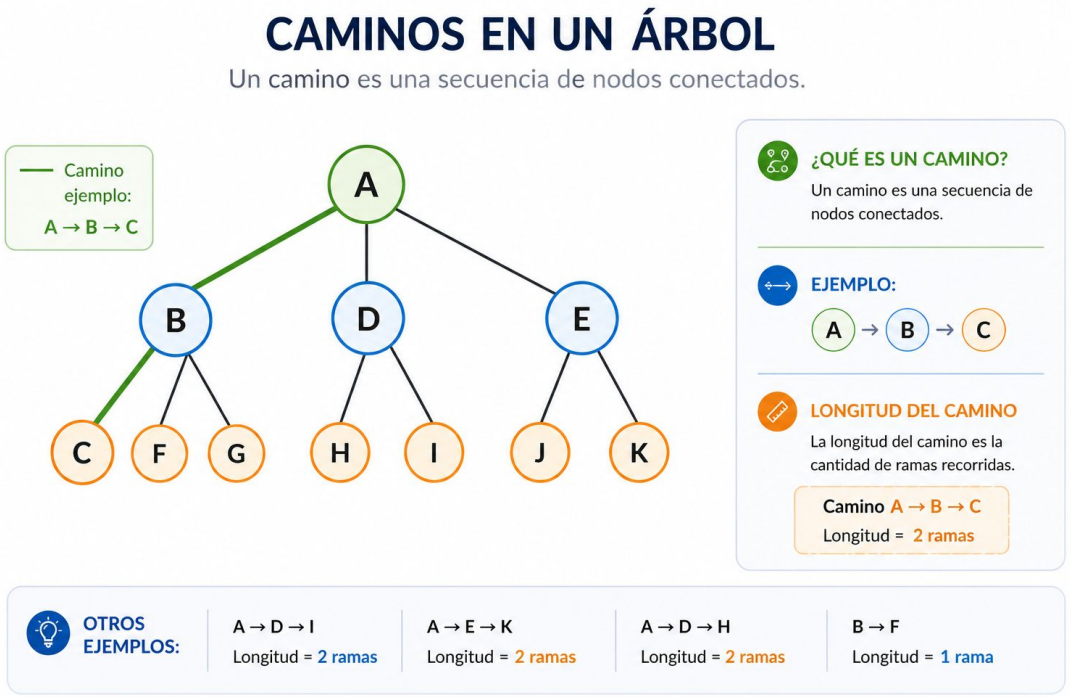


Caminos

Un camino es una secuencia de nodos conectados.

Ejemplo:
A → B → C

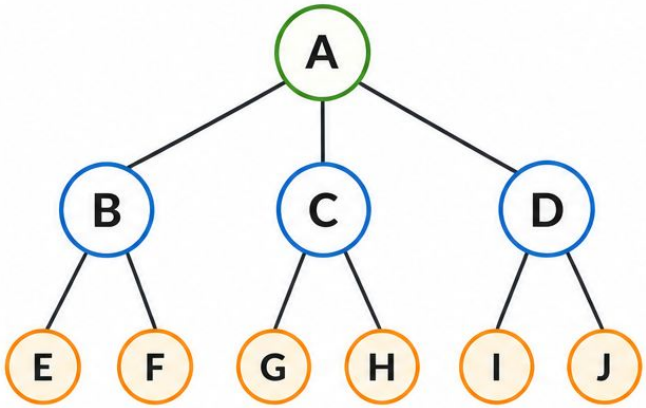
La longitud del camino es la cantidad de ramas recorridas.



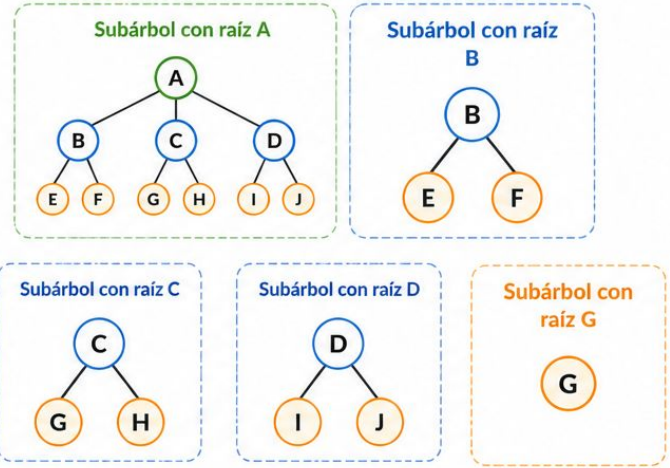
SUBÁRBOLES EN UN ÁRBOL

Cada nodo puede ser considerado la raíz de un subárbol.
Un subárbol incluye:

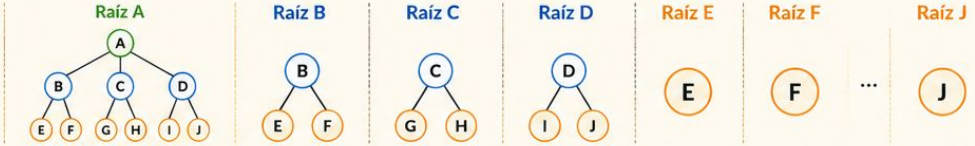
- El nodo
- Todos sus descendientes



EJEMPLOS DE SUBÁRBOLES



Idea clave:
Al elegir un nodo como raíz, tomamos ese nodo y todos los nodos que están debajo de él.



RECURSIVIDAD EN ÁRBOLES

Un árbol puede ser:

1 VACÍO

El árbol no tiene nodos.

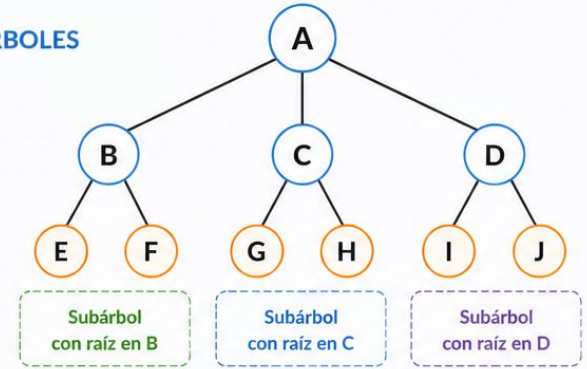
Árbol vacío

∅

2 UN NODO CON SUBÁRBOLES

El árbol está compuesto por:

- Un nodo raíz
- Cero o más subárboles

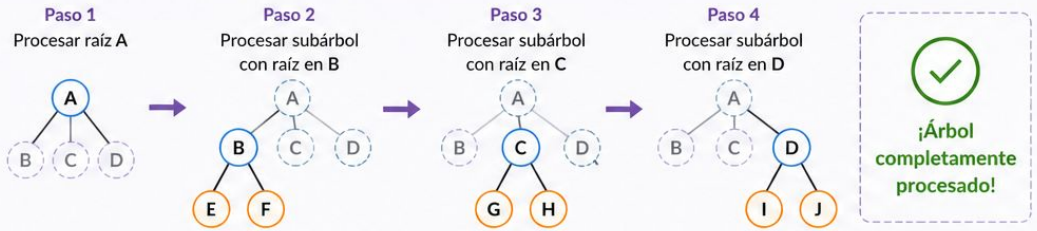


IDEA DE RECURSIVIDAD

Para procesar un árbol:

1. Si el árbol está vacío, **terminar.**
2. Si no está vacío:
   - Procesar el **nodo raíz.**
   - Aplicar el mismo procedimiento **recursivamente** a cada subárbol.

EJEMPLO DE PROCESO RECURSIVO



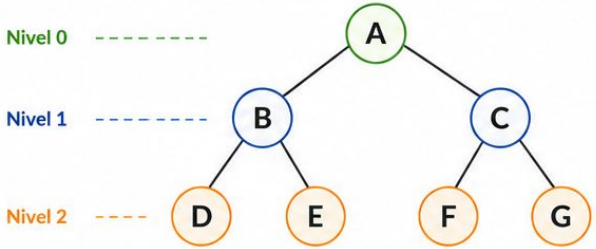
Clave: La definición recursiva de un árbol (vacío o nodo con subárboles) permite diseñar algoritmos que se aplican a sí mismos sobre partes más pequeñas del problema (los subárboles).

ÁRBOLES EQUILIBRADOS

Un árbol está equilibrado cuando:

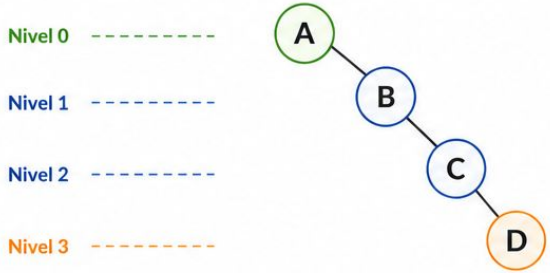
- ✓ Sus niveles están bien distribuidos
- ✓ No hay ramas mucho más largas que otras
- ✓ Mejora el rendimiento en búsquedas

ÁRBOL EQUILIBRADO



Niveles bien distribuidos.
No hay ramas desbalanceadas.

ÁRBOL NO EQUILIBRADO



Ramas mucho más largas que otras.
Desbalance en los niveles.

¿POR QUÉ ES IMPORTANTE?

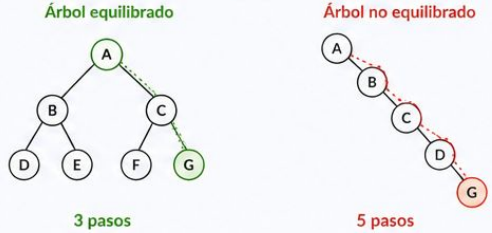
Mejora el rendimiento en búsquedas

Las búsquedas, inserciones y eliminaciones son más rápidas en árboles equilibrados.

Evita casos desfavorables

Un árbol desbalanceado puede degradarse y tener un rendimiento similar a una lista.

Ejemplo de búsqueda del nodo G

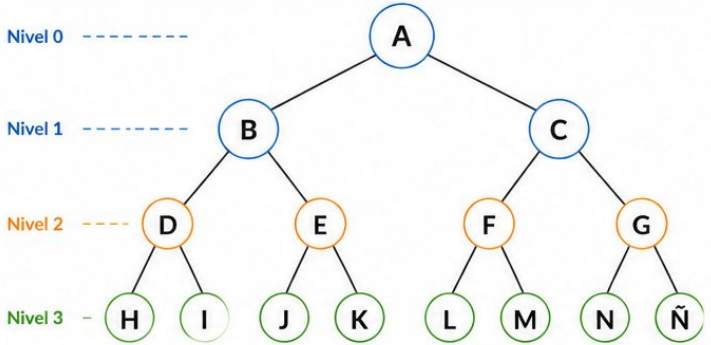


ÁRBOL PERFECTAMENTE EQUILIBRADO

Es un caso ideal donde:

- Todos los nodos tienen la misma cantidad de hijos
- Todos los niveles, excepto el último, están completamente llenos
- Las hojas se encuentran todas en el mismo nivel (o lo más cerca posible)

EJEMPLO DE ÁRBOL PERFECTAMENTE EQUILIBRADO



Todos los nodos tienen la misma cantidad de hijos (en este caso, 2).
Todos los niveles están completos y las hojas están en el mismo nivel.

¿QUÉ LO HACE IDEAL?

- Todos los nodos tienen la misma cantidad de hijos.
- Todos los niveles, excepto el último, están completamente llenos.
- Todas las hojas están en el mismo nivel o en niveles lo más cercanos posible.

RESULTADO

La altura del árbol es mínima posible para la cantidad de nodos.

¿POR QUÉ ES IMPORTANTE?

Altura mínima

La altura es mínima para la cantidad de nodos:

$h = \log_b(n)$

Búsquedas eficientes

Las búsquedas, inserciones y eliminaciones son más rápidas.

Máximo equilibrio

No hay ramas más largas que otras: el árbol está perfectamente balanceado.

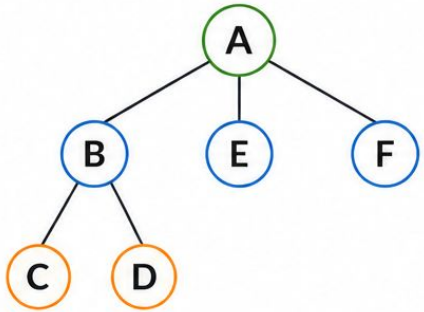
Leyenda del ejemplo:

- Nivel 0 y 1
- Nivel 2
- Nivel 3 (hojas)

REPRESENTACIÓN EN LISTAS

También se puede representar como: **lista con paréntesis**

ÁRBOL DE EJEMPLO



- Raíz
- Nodos internos
- Hojas

REPRESENTACIÓN COMO LISTA CON PARÉNTESIS

Ejemplo:

A(B(C, D), E, F)

- Cada nodo se escribe seguido de sus hijos entre paréntesis.
- Los hijos se separan por comas.
- Si un nodo no tiene hijos, no se escriben paréntesis.

¿CÓMO SE LEE?

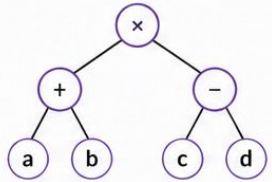
A tiene tres hijos: B, E y F.
B tiene dos hijos: C y D.
E y F no tienen hijos.

MUY USADA EN EXPRESIONES MATEMÁTICAS

Permite representar estructuras jerárquicas de forma compacta y sin ambigüedades.

EJEMPLO EN EXPRESIÓN MATEMÁTICA

Árbol de la expresión: $(a + b) \times (c - d)$



Representación en lista con paréntesis:

×(+(a, b), −(c, d))

VENTAJAS

- Compacta y fácil de escribir.
- Ideal para almacenar o transmitir árboles.
- Muy utilizada en compiladores, calculadoras simbólicas y expresiones matemáticas.

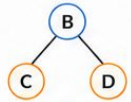
MÁS EJEMPLOS

1) Árbol con un solo nodo



A

2) Nodo con dos hijos



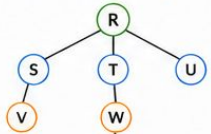
B(C, D)

3) Nodo con un hijo



X(Y)

4) Árbol más completo



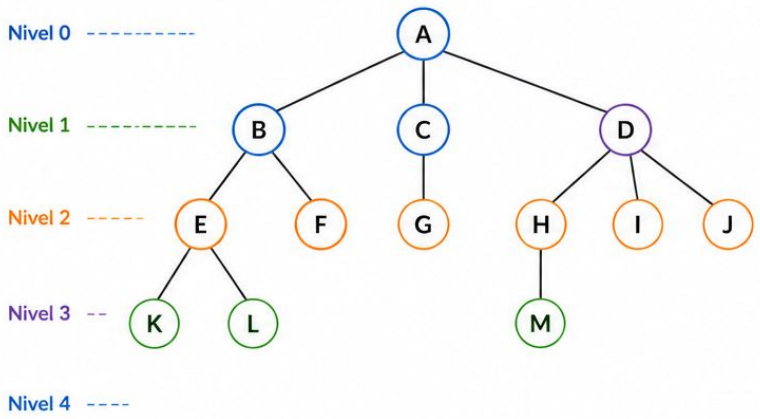
R(S(V), T(W), U)

EJEMPLO MÁS COMPLEJO

A (B (E (K , L) , F) , C (G) , D (H (M) , I , J))

Muestra múltiples niveles y subárboles

ÁRBOL REPRESENTADO



REPRESENTACIÓN EN LISTA CON PARÉNTESIS

A(B(E(K,L),F), C(G), D(H(M),I,J))

¿CÓMO SE LEE?

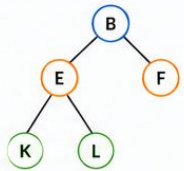
- A es la raíz y tiene tres hijos: B, C y D.
- B tiene dos hijos: E y F.
- E tiene dos hijos: K y L.
- C tiene un hijo: G.
- D tiene tres hijos: H, I y J.
- H tiene un hijo: M.
- K, L, F, G, M, I y J no tienen hijos (son hojas).

LEYENDA

- Nivel 0 y 1
- Nivel 2 y 3
- Nivel 4 (hojas)

SUBÁRBOLES PRINCIPALES

Subárbol con raíz en B



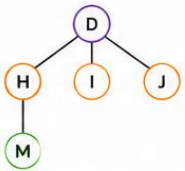
B(E(K,L),F)

Subárbol con raíz en C



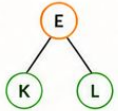
C(G)

Subárbol con raíz en D



D(H(M),I,J)

Subárbol con raíz en E

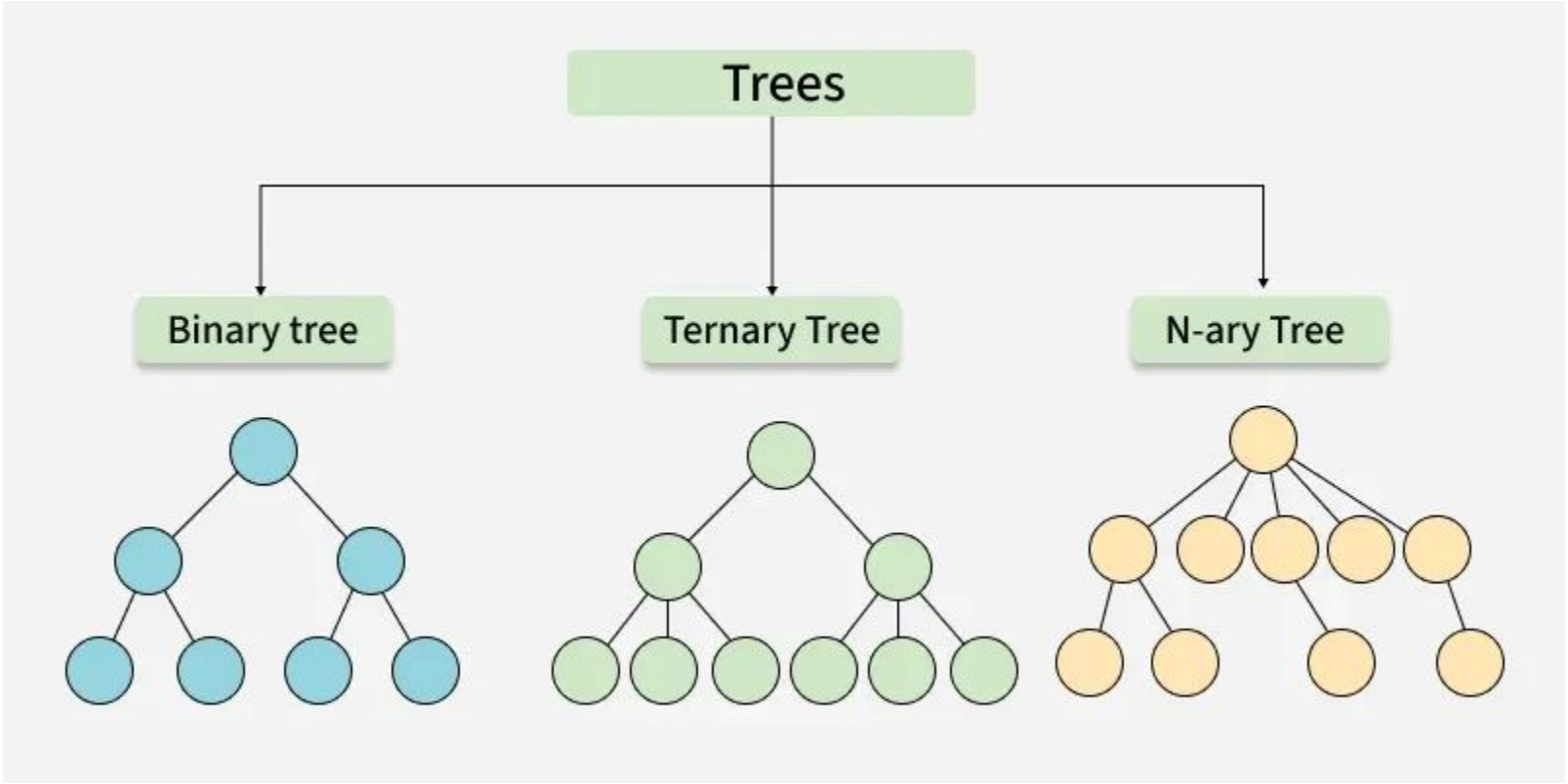


E(K,L)

CURACTERÍSTICAS DE ESTE EJEMPLO

- Tiene 5 niveles (de 0 a 4).
- Muestra distintos subárboles con diferentes tamaños.
- Incluye nodos con 1, 2 y 3 hijos.
- Las hojas están en el nivel más bajo posible dentro de cada subárbol.

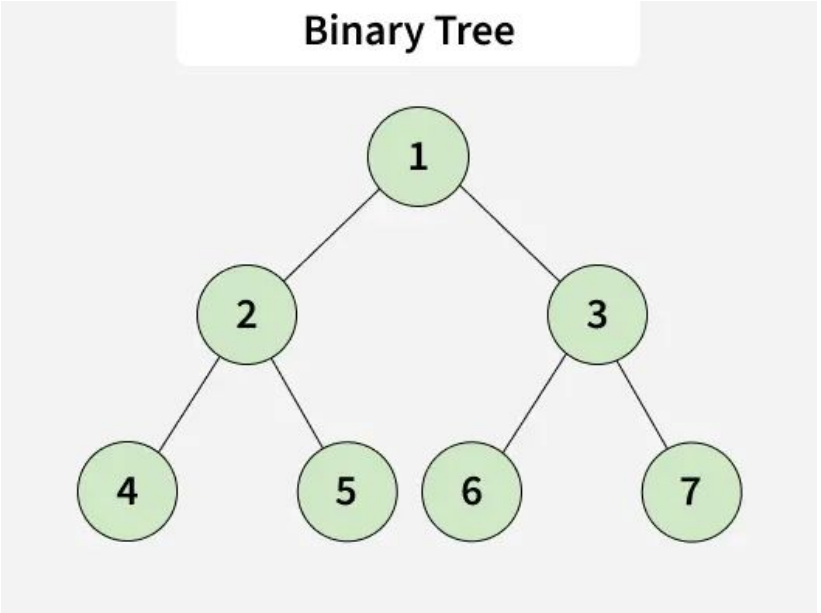
Tipos de Árboles



Árbol Binario

Un árbol binario es una estructura de datos en forma de árbol donde cada nodo tiene como máximo **dos hijos** . Estos dos hijos suelen denominarse hijo izquierdo e hijo derecho.

Consideremos el árbol . Dado que cada nodo de este árbol tiene como máximo 2 hijos, se puede decir que este árbol es un árbol binario.



Tipos de Árboles Binario

En función del número de hijos:

Árbol binario completo

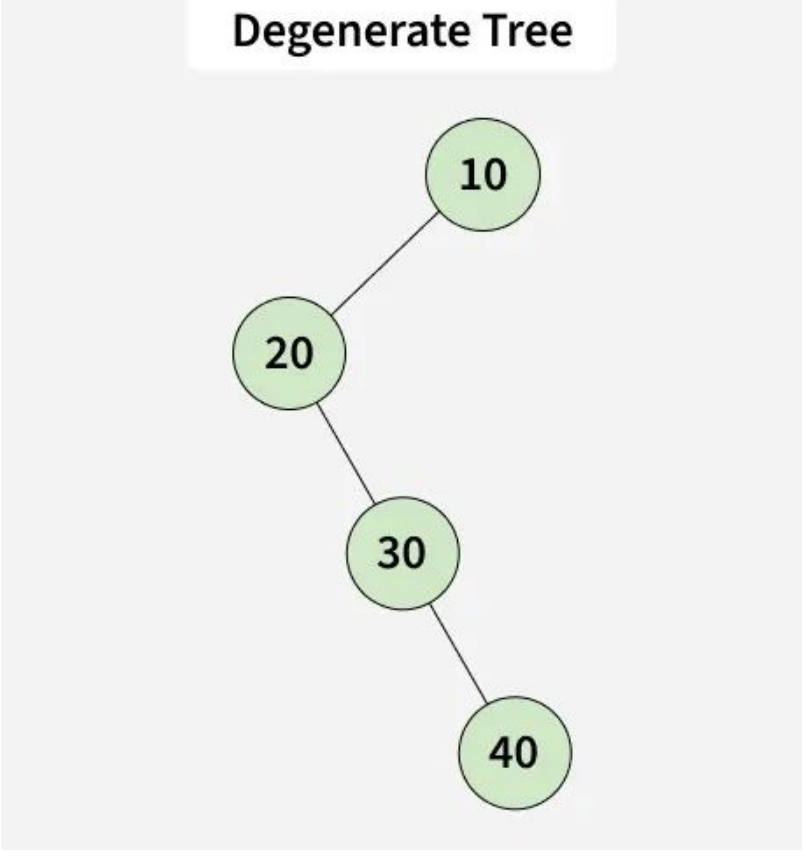
Un árbol binario es un árbol binario completo si cada nodo tiene 0 o 2 hijos. También podemos decir que un árbol binario completo es aquel en el que todos los nodos, excepto las hojas, tienen dos hijos.

Un árbol binario completo es un tipo especial de árbol binario en el que cada nodo padre/nodo interno tiene dos hijos o ninguno. También se le conoce como árbol binario propiamente dicho.

Tipos de Árboles Binario

Árbol degenerado (o patológico):

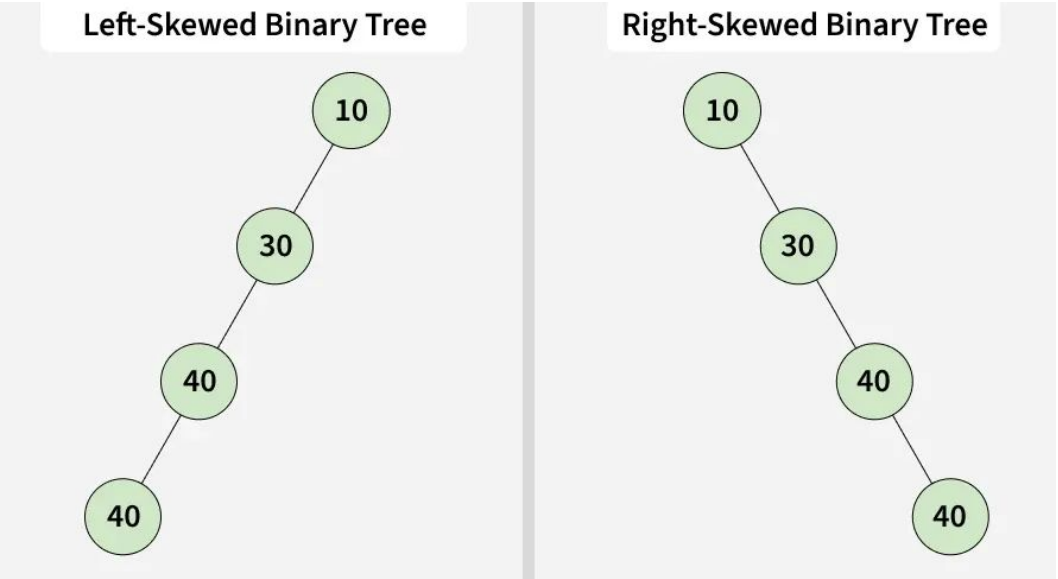
Un árbol donde cada nodo interno tiene un solo hijo. Dichos árboles tienen un rendimiento similar al de una lista enlazada. Un árbol degenerado o patológico es aquel que tiene un solo hijo, ya sea a la izquierda o a la derecha.



Tipos de Árboles Binario

Árbol binario sesgado:

Un árbol binario sesgado es un árbol patológico o degenerado en el que los nodos de la izquierda o de la derecha están dominados. Por lo tanto, existen dos tipos de árboles binarios sesgados: el sesgado a la izquierda y el sesgado a la derecha.



Tipos de Árboles Binario

En función de la finalización del nivel:

Árbol binario completo

Un árbol binario es un árbol binario completo si todos los niveles están completamente llenos, excepto posiblemente el último nivel, y este último nivel se llena desde la izquierda.

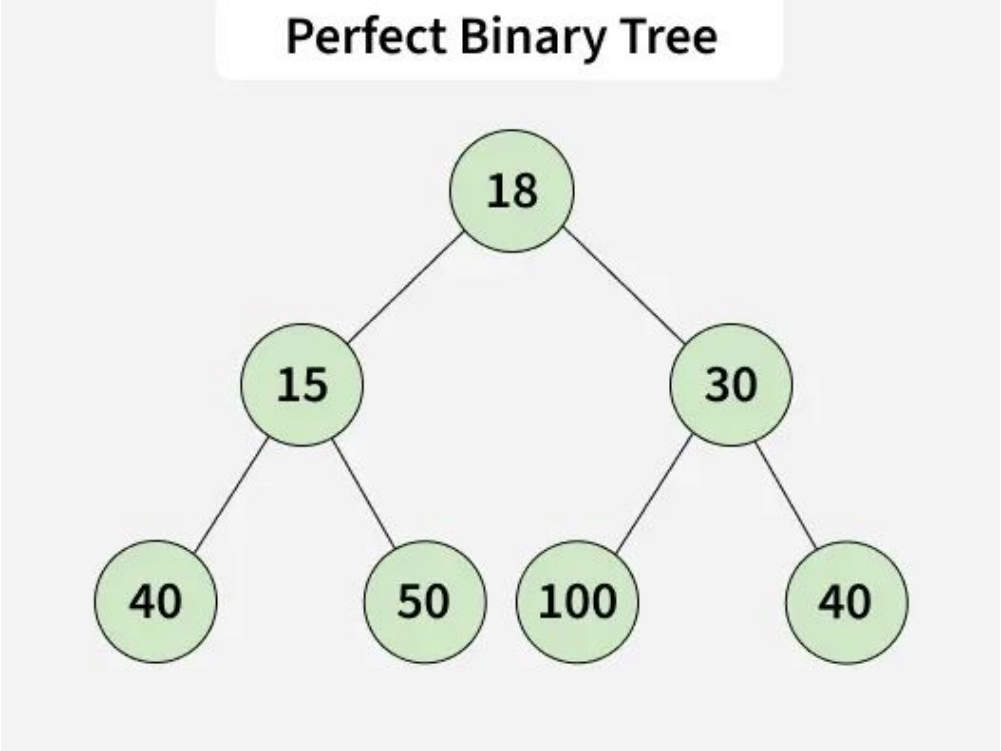
- En un árbol binario completo todas las hojas tienen la misma profundidad.
- En un árbol binario completo, el número de nodos a una profundidad **d** es $\mathbf{2^d}$.
- En un árbol binario completo con **n** nodos, la altura del árbol es **log(n+1)** .
-Todos los niveles, **excepto el último,** están completamente llenos.

Tipos de Árboles Binario

Árbol binario perfecto:

Un árbol binario perfecto es aquel en el que todos los nodos internos tienen dos hijos y todos los nodos hoja están al mismo nivel.

En un árbol binario perfecto, el número de nodos hoja es igual al número de nodos internos más uno.

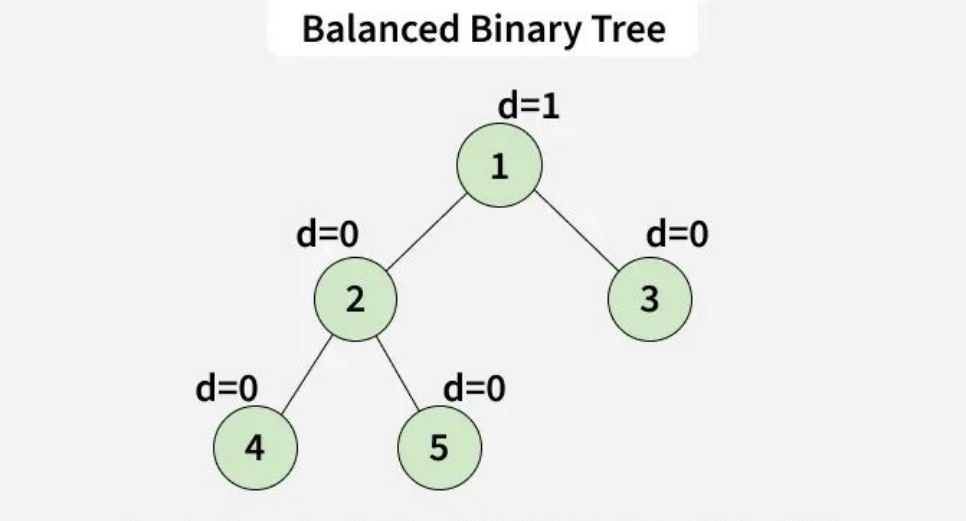


Tipos de Árboles Binario

Árbol binario equilibrado:

Un árbol binario está equilibrado si la altura del árbol es O(log n), donde n es el número de nodos.

Por ejemplo, el árbol AVL mantiene una altura de O(log n) asegurándose de que la diferencia entre las alturas de los subárboles izquierdo y derecho de cada nodo sea como máximo 1. *(Factor de Balance)*

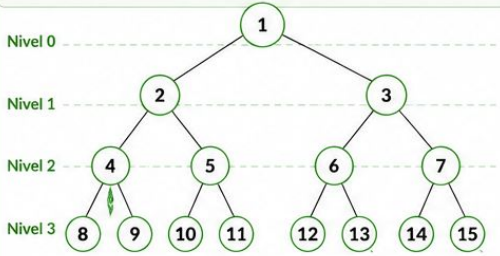


COMPARACIÓN: ÁRBOL PERFECTO, COMPLETO Y EQUILIBRADO

Todos son árboles binarios, pero tienen **diferentes propiedades.**

ÁRBOL PERFECTO

Todos los niveles están completamente llenos y todas las hojas están en el mismo nivel.



Características

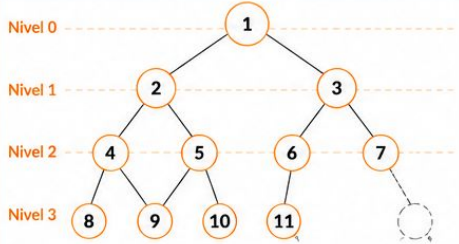
- Todos los niveles llenos al 100%.
- Número de nodos: $2^{h+1} - 1$ (donde h es la altura).
- Todas las hojas en el mismo nivel.
- Es un caso particular de árbol completo y equilibrado.

Ejemplo con altura 3 (h = 3)

Nodos = $2^{3+1} - 1 = 15$

ÁRBOL COMPLETO

Todos los niveles están completamente llenos, excepto posiblemente el último, que se llena de izquierda a derecha.



Características

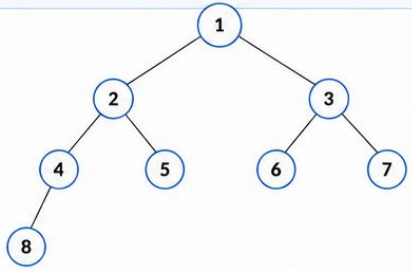
- Todos los niveles llenos, excepto el último.
- El último nivel se llena de izquierda a derecha sin dejar huecos.
- No necesariamente todas las hojas están en el mismo nivel.
- No siempre es equilibrado.

Ejemplo con 11 nodos

El último nivel no está completo, pero se llena de izquierda a derecha.

ÁRBOL EQUILIBRADO

Para cada nodo, la diferencia entre la altura de su subárbol izquierdo y derecho es como máximo 1.



Características

- En cada nodo: |altura(izquierdo) − altura(derecho)| ≤ 1.
- La altura del árbol es O(log n).
- No todos los niveles deben estar llenos.
- Puede no ser completo ni perfecto.

Ejemplo

Este árbol está equilibrado:

- ☑ En todos los nodos la diferencia de alturas entre subárboles es ≤ 1.
- ☑ Pero no es completo (el último nivel no se llena de izquierda a derecha) y no es perfecto (no todos los niveles están llenos).

RESUMEN RÁPIDO

	ÁRBOL PERFECTO	ÁRBOL COMPLETO	ÁRBOL EQUILIBRADO
Propiedad principal	Todos los niveles llenos	Último nivel lleno de izquierda a derecha	Diferencia de alturas ≤ 1 en cada nodo
Último nivel	Siempre lleno	Puede no estar lleno (pero sin huecos a la izquierda)	Puede tener huecos en cualquier posición
Todas las hojas al mismo nivel	Sí	No necesariamente	No necesariamente
Ejemplo	Caso ideal ($2^{h+1} - 1$ nodos)	Uso eficiente del espacio	Árboles balanceados como AVL, Red-Black, etc.

Tipos de Árboles Binario

Árbol de búsqueda binaria (BST) y sus variaciones : Un BST es un árbol binario donde cada nodo tiene como máximo dos hijos, y para cada nodo, el valor del hijo izquierdo es menor que el valor del nodo, y el valor del hijo derecho es mayor.

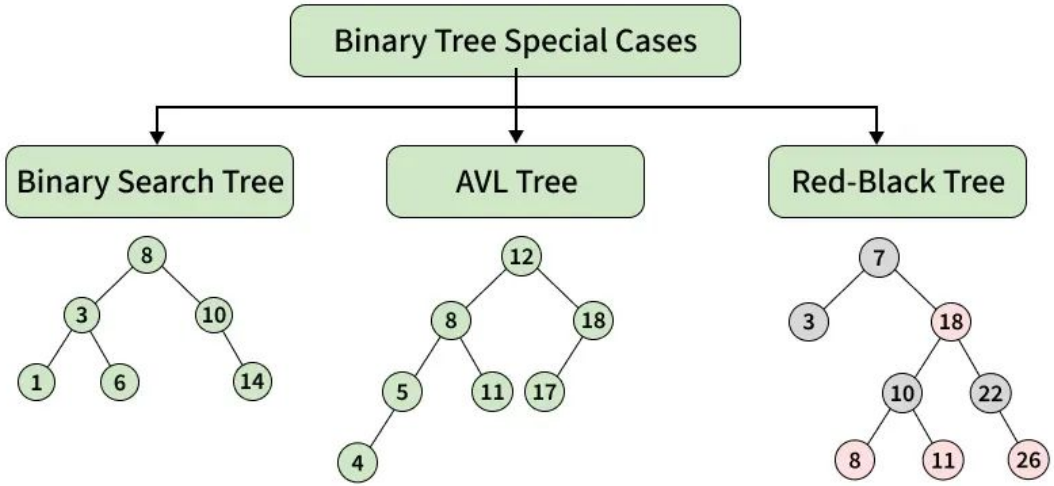
Árbol binario indexado (árbol de Fenwick): una estructura de datos que utiliza un árbol binario para calcular y actualizar de forma eficiente las sumas de prefijos en una matriz.

Árbol binario equilibrado : Un árbol binario donde la diferencia de alturas entre los subárboles izquierdo y derecho de cualquier nodo es mínima (a menudo definida como máximo 1). Ejemplos de árboles binarios equilibrados son el árbol AVL, el árbol rojo-negro y el árbol Splay .

Algunos tipos especiales de árboles

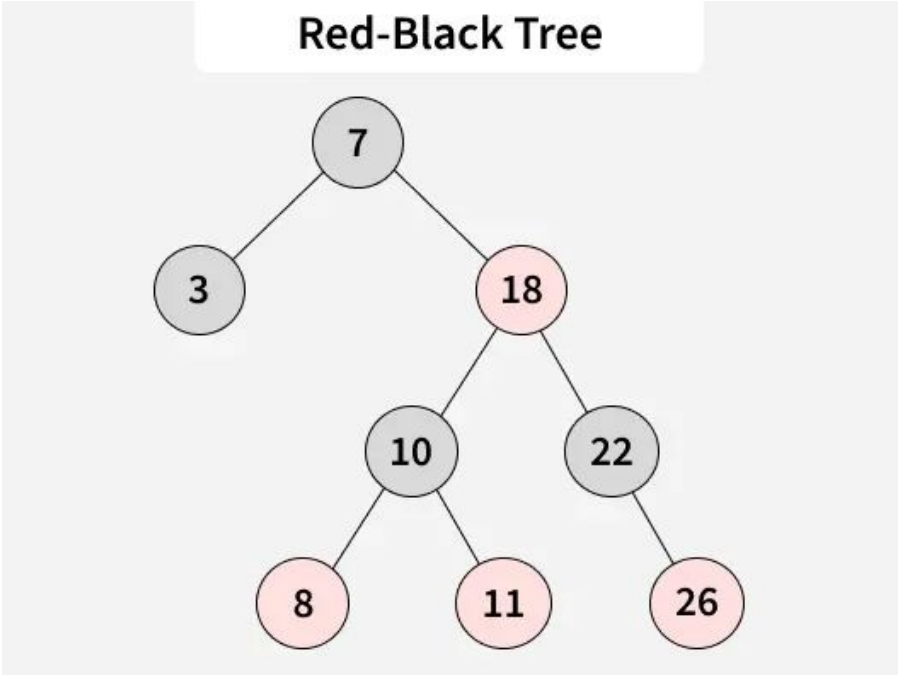
En función de los valores de los nodos, el árbol binario se puede clasificar en los siguientes tipos especiales:

- Árbol rojo y negro
- Árbol B
- Árbol B+
- **Árbol de búsqueda binaria**
- **Árbol AVL**



Árbol rojo y negro

Un árbol rojo-negro es un tipo de árbol de búsqueda binaria autoequilibrado donde cada nodo tiene un **bit adicional** , que suele interpretarse como el **color** (rojo o negro). Estos colores se utilizan para asegurar que el árbol se mantenga equilibrado durante las inserciones y eliminaciones. Si bien el equilibrio del árbol no es perfecto, es suficiente para reducir el tiempo de búsqueda y mantenerlo en torno a O(log n), donde n es el número total de elementos en el árbol.



Árbol - B

Un árbol B es un árbol de búsqueda autoequilibrado diseñado para operaciones eficientes de inserción, eliminación y búsqueda en sistemas basados en disco. Cada nodo puede tener múltiples claves e hijos; las claves dentro de un nodo están ordenadas y los subárboles entre claves almacenan valores dentro de ese rango. Esta estructura garantiza el equilibrio y minimiza la altura para un acceso rápido, incluso con grandes conjuntos de datos.

Operaciones en el árbol B

Los árboles B admiten diversas operaciones que los hacen altamente eficientes para gestionar grandes conjuntos de datos.

Operación	**Complejidad temporal**
Buscar	O(log n)
Insertar	O(log n)
Borrar	O(log n)
Recorrer	O(n)

ÁRBOL B (B-TREE)

Un **Árbol B** es un árbol balanceado de búsqueda en el que cada nodo puede tener más de 2 hijos y contiene múltiples claves ordenadas. Está diseñado para minimizar accesos a disco y se usa en bases de datos e índices.

PARÁMETRO PRINCIPAL

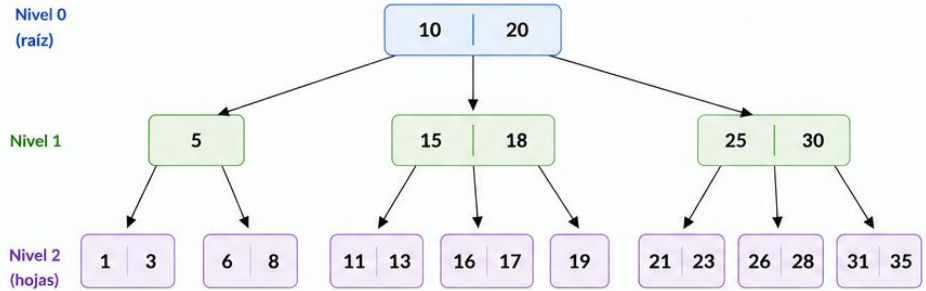
t: grado mínimo del árbol ($t \geq 2$)

- Cada nodo puede tener como mínimo t-1 claves y como máximo $2t$-1 claves.
- Cada nodo puede tener como mínimo t hijos (no raíz) y como máximo $2t$ hijos.

En este ejemplo:
$t = 2$

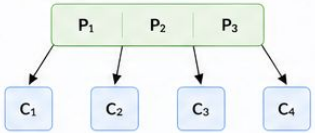
- Mínimo claves por nodo: 1
- Máximo claves por nodo: 3
- Mínimo hijos por nodo (no raíz): 2
- Máximo hijos por nodo: 4

EJEMPLO DE ÁRBOL B (t = 2)



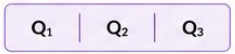
ESTRUCTURA DE UN NODO EN UN ÁRBOL B (t = 2)

Nodo interno



- Contiene entre 1 y 3 claves ($2t$-1 = 3 claves máx.)
- Contiene entre 2 y 4 hijos ($2t$ = 4 hijos máx.)
- Si tiene k claves, entonces tiene $k+1$ hijos.

Nodo hoja



- Contiene entre 1 y 3 claves.
- No tiene hijos.

PROPIEDADES DE UN ÁRBOL B (grado mínimo t)

1. Cada nodo contiene entre t-1 y $2t$-1 claves.
2. Cada nodo (excepto la raíz) tiene entre t y $2t$ hijos.
3. La raíz tiene entre 1 y $2t$-1 claves y al menos 2 hijos si no es hoja.
4. Todas las hojas están en el mismo nivel.
5. Las claves dentro de un nodo están ordenadas.
6. Para un nodo con claves $[k_1, k_2, ..., k_m]$ y sus $m+1$ hijos $[C_1, C_2, ..., C_{m+1}]$:
   Todas las claves de $C_1 < k_1$
   Todas las claves de C_i están entre k_{i-1} y k_i $(1 < i \leq m)$
   Todas las claves de $C_{m+1} > k_m$

OPERACIONES PRINCIPALES

BÚSQUEDA
Se busca la clave comparándola con las claves del nodo actual y descendiendo al hijo adecuado.

INSERCIÓN
Se inserta en la hoja adecuada. Si un nodo se desborda (tiene $2t$ claves), se divide.

ELIMINACIÓN
Se elimina la clave. Si un nodo tiene menos de t-1 claves, se redistribuye o se fusiona.

ANALOGÍA

Imagina cada nodo como una página de un libro donde pueden guardarse varias claves. Los hijos son los punteros a las páginas siguientes.



EN RESUMEN: Un **Árbol B** mantiene los datos balanceados permitiendo que cada nodo almacene múltiples claves y tenga múltiples hijos, lo que reduce la altura del árbol y mejora el rendimiento en sistemas que usan almacenamiento en disco.

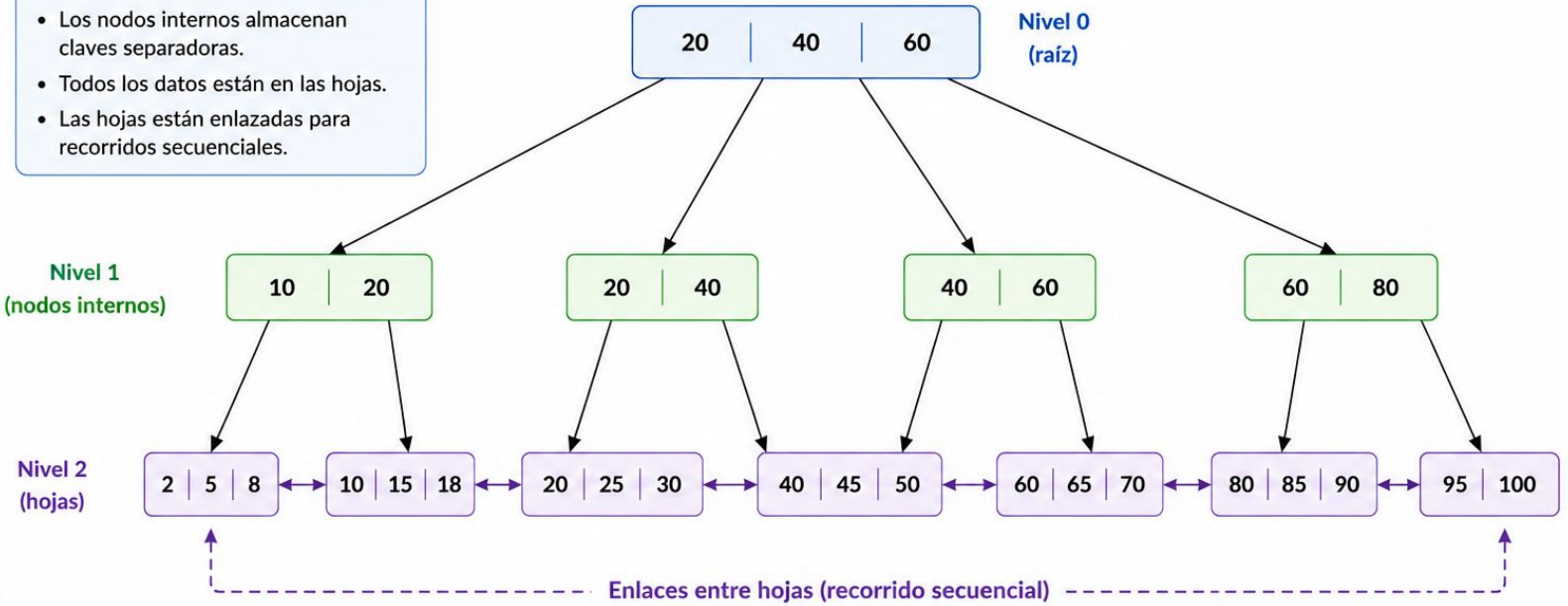
Árbol B+

Un árbol B+ es una extensión del árbol B en la que todos los datos se almacenan únicamente en los nodos hoja, mientras que los nodos internos almacenan las claves solo para la indexación. Los nodos hoja se enlazan secuencialmente, formando una lista enlazada que permite realizar consultas de rango eficientes y recorridos ordenados.

Es una estructura de datos avanzada que se utiliza en sistemas de bases de datos y sistemas de archivos para mantener los datos ordenados y facilitar su recuperación rápida, especialmente desde el disco.

ÁRBOL B+ (orden 4)

- Orden 4 → cada nodo puede tener hasta 4 hijos.
- Los nodos internos almacenan claves separadoras.
- Todos los datos están en las hojas.
- Las hojas están enlazadas para recorridos secuenciales.



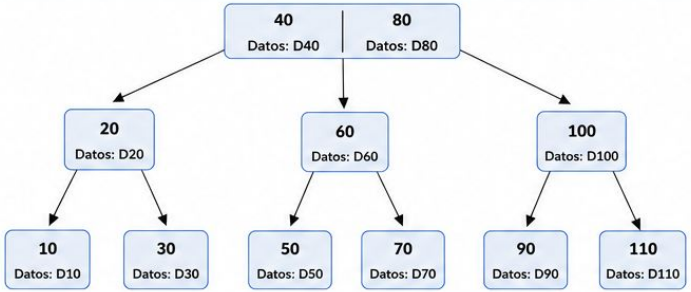
CARACTERÍSTICAS DEL ÁRBOL B+

- Todos los datos están en las hojas.
- Los nodos internos solo guían la búsqueda.
- Las hojas están enlazadas entre sí.
- Ideal para búsquedas por rango.

DIFERENCIA ENTRE ÁRBOL B Y ÁRBOL B+

ÁRBOL B (mezcla de claves y datos)

- Los datos pueden estar en cualquier nodo (interno o hoja).
- Los nodos internos también guardan datos.



Nodo interno (contiene claves y datos)

Hoja (contiene claves y datos)

Recorrido en orden (no es tan eficiente)

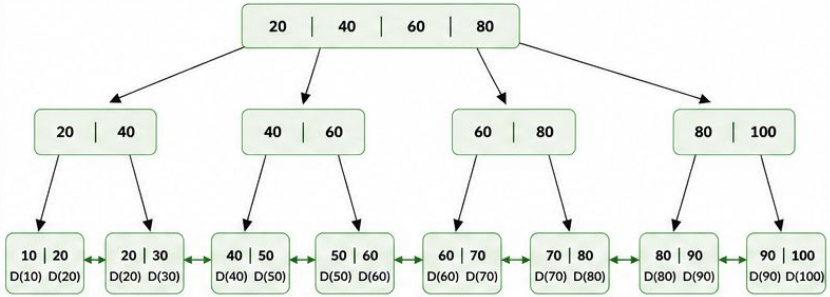


Características del Árbol B

- ✓ Los datos pueden estar en nodos internos y en hojas.
- ✓ Puede encontrarse el dato antes de llegar a una hoja.
- ✓ Menos repetición de claves.
- ✓ Recorridos secuenciales menos eficientes (salvo que se recorran todas las hojas).

ÁRBOL B+ (datos solo en hojas, claves repetidas para indexación)

- Los nodos internos NO guardan datos, solo claves (índices).
- Todos los datos están únicamente en las hojas.
- Las hojas están enlazadas secuencialmente.



Nodo interno (solo claves / índices)

Hoja (solo datos)

Recorrido en orden (muy eficiente)



Características del Árbol B+

- ✓ Los internos solo indexan, los datos están solo en las hojas.
- ✓ Siempre se llega a una hoja para encontrar el dato.
- ✓ Claves repetidas: las claves que indexan a las hojas se repiten en las hojas.
- ✓ Recorridos secuenciales muy eficientes gracias al enlace de las hojas.
- ✓ Ideal para consultas por rango.

Ejemplo de claves repetidas (observa que 20, 40, 60, 80 aparecen arriba y también en las hojas)

COMPARACIÓN RÁPIDA	ÁRBOL B	ÁRBOL B+
¿Dónde están los datos?	En nodos internos y hojas	Solo en las hojas
¿Qué guardan los nodos internos?	Claves y datos	Solo claves (índices)
¿Se repiten las claves?	No necesariamente	Sí, para indexar las hojas
¿Se puede encontrar el dato antes de llegar a hoja?	Sí	No, siempre en hoja
¿Recorrido secuencial?	Menos eficiente	Muy eficiente (hojas enlazadas)
¿Uso típico?	Sistemas de archivos, índices más generales	Bases de datos, consultas por rango, sistemas de almacenamiento

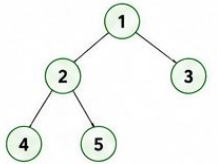
Idea clave

El Árbol B es una mezcla de datos + índice.

El Árbol B+ separa índice arriba y datos abajo (en hojas), optimizando búsquedas y recorridos por rango.

1. ÁRBOLES BÁSICOS (Estructuras fundamentales)

ÁRBOL BINARIO

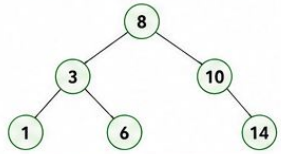


- Cada nodo tiene máximo 2 hijos.

Operaciones: Recorrer, Insertar, Eliminar

Complejidad: O(n) – O(log n)*

ÁRBOL BINARIO DE BÚSQUEDA (ABB)

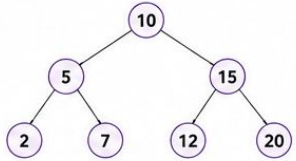


- Izq < raíz < der.
- Búsqueda eficiente.

Operaciones: Buscar, Insertar, Eliminar

Complejidad: Promedio: O(log n) Peor caso: O(n)

AVL (ÁRBOL AVL)

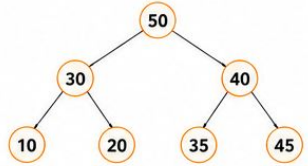


- ABB balanceado.
- Factor de balance: -1, 0 o 1.

Operaciones: Buscar, Insertar, Eliminar + Rotaciones

Complejidad: O(log n)

HEAP (MONTÍCULO)



- Árbol completo.
- Propiedad de heap (máx. o mín.).

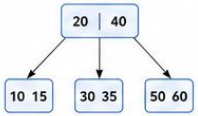
Operaciones: Insertar, Extraer máx/mín

Complejidad: O(log n)

* O(log n) cuando el árbol está balanceado.

2. ÁRBOLES PARA BASES DE DATOS E ÍNDICES

ÁRBOL B

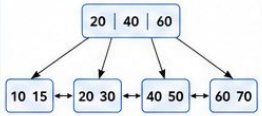


- Datos y claves en nodos internos y hojas.
- Balanceado de orden m.

Usos: Sistemas de archivos, índices en disco

Complejidad: $O(\log_m n)$

ÁRBOL B+

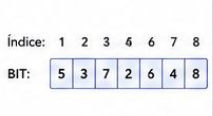


- Internos: solo índices.
- Datos solo en hojas.
- Hojas enlazadas.

Usos: Bases de datos, índices, consultas por rango

Complejidad: $O(\log_m n)$

ÁRBOL FENWICK (BIT)

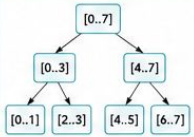


- Estructura de arreglo.
- Sumas prefijo y actualizaciones.
- Muy eficiente en memoria.

Usos: Suma de prefijos, inversiones, consultas acumulativas

Complejidad: O(log n)

SEGMENT TREE



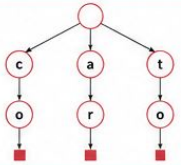
- Divide en segmentos.
- Consultas de rango.
- Actualizaciones puntuales.

Usos: Consultas de rango (mín, máx, suma, gcd, etc.)

Complejidad: O(log n)

3. ÁRBOLES PARA TEXTO Y CADENAS

TRIE (ÁRBOL PREFIJO)

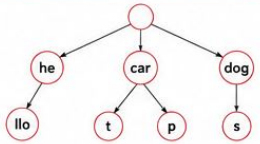


- Almacena palabras por prefijos.
- Búsqueda, inserción y prefijos.
- Autocompletado.

Usos: Diccionarios, autocompletado, búsqueda de palabras

Complejidad: O(L) (L = longitud de la palabra)

PATRICIA TRIE (ÁRBOL RADIX COMPACTO)



- Trie comprimido.
- Menos nodos, más eficiente en memoria.
- Comparte prefijos comunes.

Usos: Diccionarios grandes, IP routing, compresión de prefijos

Complejidad: O(L)

Árboles básicos (estructuras fundamentales)

Índices y bases de datos (almacenamiento eficiente)

Texto y cadenas (prefijos y diccionarios)

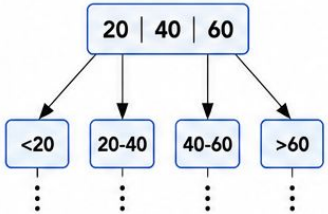
Prioridad / Montículos (colas de prioridad)

Consultas y análisis (rangos y acumulaciones)

2 ÁRBOLES PARA ÍNDICES Y BASES DE DATOS

Estructuras pensadas para indexación, consultas, rangos y grandes volúmenes de datos

ÁRBOL B

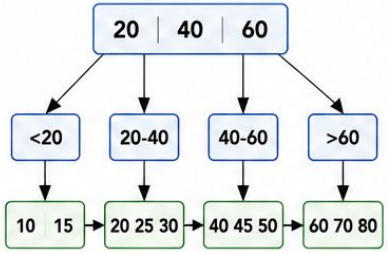


- Múltiples claves por nodo.
- Todos los datos están en los nodos (internos y hojas).
- Balanceado de orden m.

USOS
Sistemas de archivos, índices en disco.

COMPLEJIDAD
$O(\log_m n)$

ÁRBOL B+

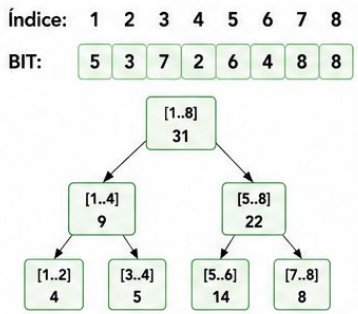


- Internos: solo índices.
- Todos los datos están en las hojas.
- Hojas enlazadas (rango eficiente).

USOS
Bases de datos, índices secundarios, consultas por rango.

COMPLEJIDAD
$O(\log_m n)$

FENWICK TREE (BIT)

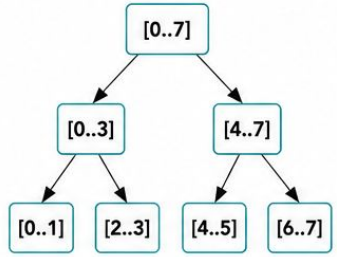


- Estructura de arreglo.
- Suma de prefijos y consultas acumulativas.
- Muy eficiente en memoria.

USOS
Estadísticas, inversiones, consultas acumulativas.

COMPLEJIDAD
$O(\log n)$

SEGMENT TREE



- Divide en segmentos.
- Consultas de rango.
- Actualizaciones puntuales.

USOS
Consultas de rango (mín, máx, suma, gcd, etc.).

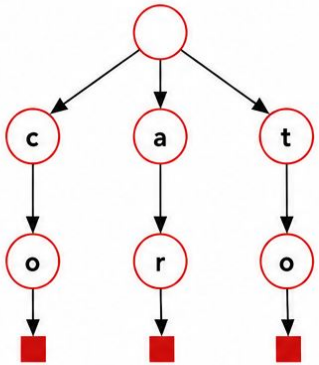
COMPLEJIDAD
$O(\log n)$

IDEALES PARA: indexar datos, gestionar grandes volúmenes, consultas por rango y trabajar con datos en disco o memoria.

3 ÁRBOLES PARA TEXTO Y CADENAS

Estructuras especializadas en palabras, prefijos y autocompletado

TRIE (ÁRBOL PREFIJO)



- Almacena palabras por prefijos.
- Búsqueda, inserción y prefijos.
- Autocompletado.

USOS

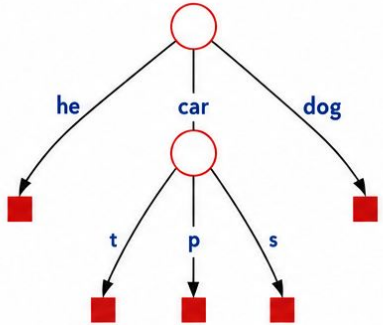
- Diccionarios
- Autocompletado
- Búsqueda de palabras

COMPLEJIDAD

O(L)

L = longitud de la palabra

PATRICIA TRIE (ÁRBOL RADIX COMPACTO)



- Trie comprimido.
- Menos nodos, más eficiente en memoria.
- Comparte prefijos comunes.

USOS

- Diccionarios grandes
- IP routing (rutas de red)
- Compresión de prefijos

COMPLEJIDAD

O(L)

L = longitud de la palabra

IDEALES PARA: trabajar con cadenas de caracteres, prefijos y búsquedas rápidas (autocompletado).

4 COMPARACIÓN VISUAL FINAL

Resumen rápido para elegir la estructura adecuada

ÁRBOL / ESTRUCTURA	USO PRINCIPAL	DATOS	CONSULTAS SOPORTADAS	COMPLEJIDAD (BÚSQUEDA)	IDEAL PARA
AVL (Árbol AVL)	Búsqueda balanceada	En memoria	Buscar / Insertar / Eliminar	O(log n)	Aplicaciones generales con muchas búsquedas
Heap (Montículo)	Cola de prioridad	En memoria	Insertar / Extraer máx o mín	O(log n)	Schedulers, colas de prioridad
B+ Tree (Árbol B+)	Índices y bases de datos	En disco	Buscar / Rango / Insertar / Eliminar	$O(\log_m n)$	Bases de datos, índices, sistemas de archivos
Fenwick Tree (BIT)	Suma de prefijos y consultas acumulativas	En memoria	Prefijo / Punto	O(log n)	Estadísticas, inversiones, acumulaciones
Segment Tree	Consultas de rango	En memoria	Rango / Punto	O(log n)	Mín, máx, suma, gcd, actualizaciones
Trie (Árbol Prefijo)	Diccionarios y búsqueda por prefijo	En memoria	Buscar / Insertar / Prefijos	O(L)	Autocompletado, búsqueda de palabras
Patricia Trie (Radix)	Diccionarios grandes y prefijos	En memoria	Buscar / Insertar / Prefijos	O(L)	IP routing, compresión de prefijos

REFERENCIAS

- n = cantidad de elementos
- m = orden del árbol B/B+
- L = longitud de la palabra

REGLA RÁPIDA

Si es por rango → B+ Tree o Segment Tree

Si es por prefijos → Trie o Patricia Trie

Si es búsqueda general → AVL

IDEALES PARA: elegir rápidamente la estructura correcta según tus datos, consultas y restricciones.

Equilibrio en Árboles Binarios

La distancia entre un nodo y la raíz influye directamente en la rapidez con la que puede encontrarse dentro del árbol. Cuanto más cerca esté un nodo de la raíz, menos recorridos o bifurcaciones se necesitan para acceder a él.

A partir de esto surge una característica fundamental de los árboles binarios: el equilibrio o balance. Un árbol equilibrado permite búsquedas más eficientes, ya que distribuye sus nodos de manera más uniforme.

Para medir este equilibrio se utiliza el **factor de equilibrio**, que representa la diferencia de altura entre el subárbol derecho y el izquierdo:

B = hD - hI

donde **hD** es la altura del subárbol derecho y **hI** la del izquierdo.

Equilibrio en Árboles Binarios

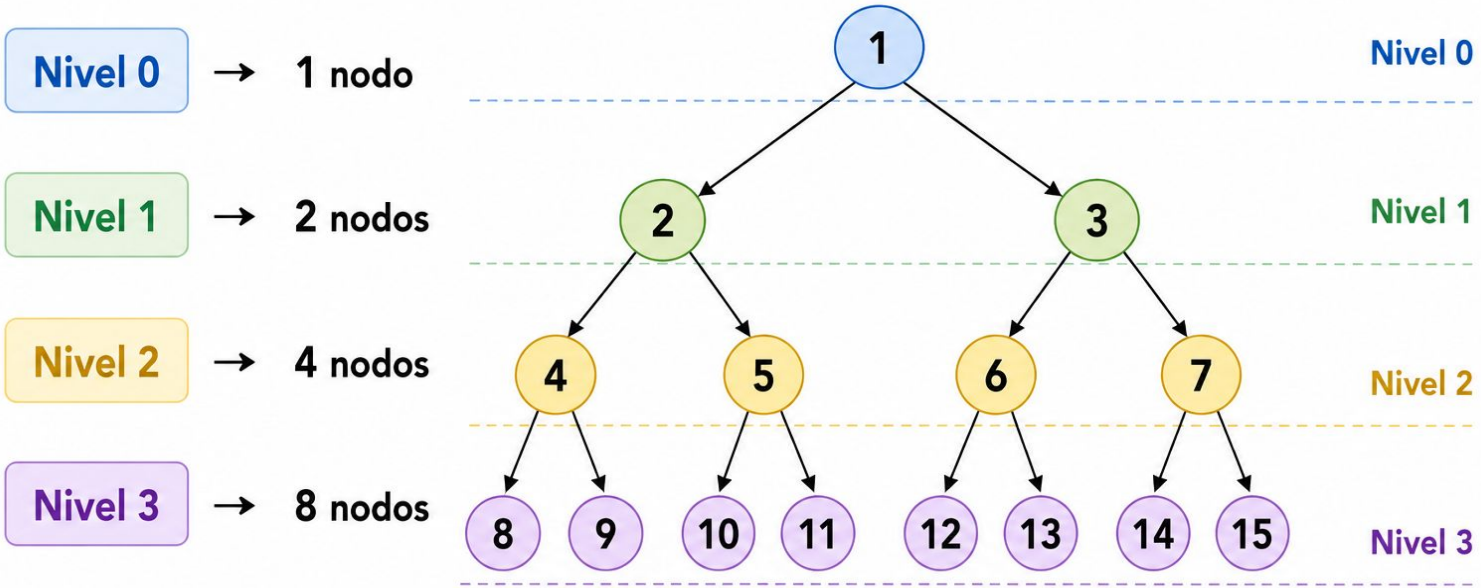
Un árbol binario se considera perfectamente equilibrado cuando su factor de equilibrio es igual a cero y sus subárboles también mantienen ese equilibrio.

Sin embargo, como esta situación es poco frecuente, se utiliza una definición más flexible: un árbol está equilibrado cuando la diferencia de altura entre sus subárboles izquierdo y derecho no supera una unidad.

Por esta razón, el factor de equilibrio de cada nodo puede tomar únicamente los valores:

-1, 0 o +1.

Crecimiento de un árbol binario



Cada nivel duplica la cantidad de nodos del anterior.

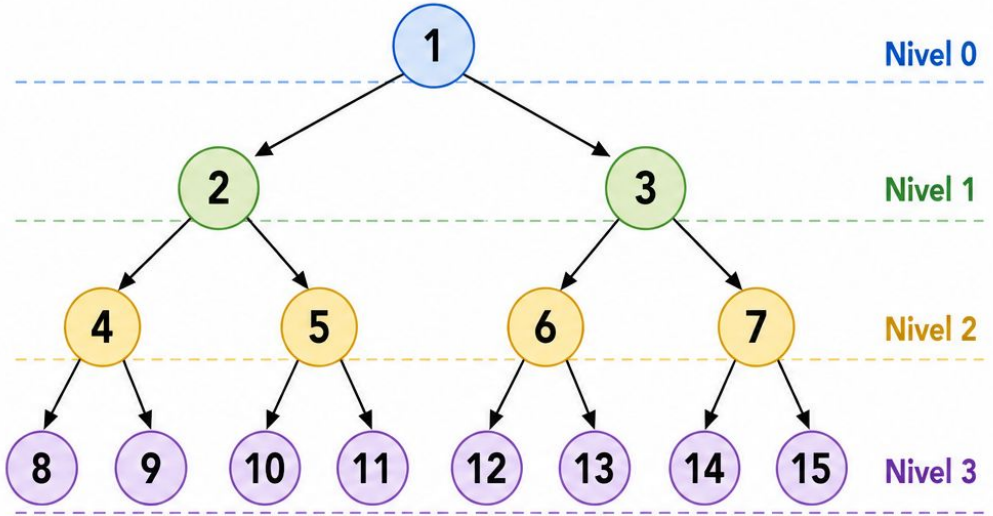
En un árbol binario lleno, el total de nodos se calcula con:

$$2^{(k+1)} - 1$$

EJEMPLO:

$2^4 - 1 = 15$ nodos

Donde **k** es la cantidad de niveles (comenzando desde el nivel 0).



Para $k = 3$ niveles (0, 1, 2, 3): $2^{(3+1)} - 1 = 2^4 - 1 = 16 - 1 =$ **15 nodos**

Altura del árbol y complejidad algorítmica

En un árbol binario equilibrado, la altura crece de forma logarítmica respecto a la cantidad de nodos.

Esto significa que para encontrar un elemento no es necesario recorrer todos los nodos, sino solo una pequeña cantidad de niveles.

15 nodos → altura ≈ 4

1.000 nodos → altura ≈ 10

1.000.000 nodos → altura ≈ 20

Aunque los datos crezcan muchísimo, la cantidad de pasos necesarios aumenta muy poco.

Operaciones en árboles binarios

Los árboles binarios permiten realizar distintas operaciones, como calcular su altura, contar nodos, copiar estructuras, comparar árboles, eliminar nodos o evaluar expresiones matemáticas.

Para ejecutar estas tareas se utiliza el **recorrido de árboles**, que consiste en visitar cada nodo una sola vez siguiendo un orden específico.

Los recorridos permiten procesar la información almacenada en la estructura, por ejemplo para realizar búsquedas, mostrar datos, ordenar información o evaluar expresiones.

Existen diferentes tipos de recorridos, cada uno orientado a resolver distintos problemas de manera eficiente.

Árboles de expresión

Un árbol de expresión es un árbol binario utilizado para representar expresiones matemáticas o lógicas en memoria.

En este tipo de árboles:

- Las **hojas** representan operandos *(números o variables)*.
- Los **nodos internos** representan operadores (*,+, -, , /, etc.)*.
- Cada subárbol representa una subexpresión.

Ejemplo de expresión infija: **a * (b + c) + d**

Los paréntesis no aparecen explícitamente en el árbol, pero la estructura del árbol conserva el orden correcto de evaluación.

Uso principal: Compiladores, intérpretes y evaluación de expresiones.

Representación en un árbol binario

Cada operador se ubica en un nodo raíz y sus operandos se almacenan en los subárboles izquierdo y derecho.

(x + y) * (a - b)

Interpretación:

- La raíz contiene el operador principal (*).
- El subárbol izquierdo representa la subexpresión (x + y).
- El subárbol derecho representa la subexpresión (a - b).
- Las hojas contienen los operandos.

Ventaja: La estructura facilita el análisis y la evaluación automática de expresiones.

Reglas para construir árboles de expresión

Un árbol de expresión se construye analizando la prioridad de los operadores y agrupando las operaciones mediante paréntesis.

La idea principal es que cada operación importante se convierte en un nodo del árbol y sus operandos pasan a formar parte de los subárboles izquierdo y derecho.

a * c + e / g - (b + d)

Primero se resuelven las operaciones de mayor prioridad:
(a * c) (e / g) (b + d)
Luego se combinan las expresiones restantes de izquierda a derecha:
((a * c) + (e / g)) - (b + d)

Finalmente, toda la expresión queda organizada en un único árbol..

Recorrido de árboles

Recorrer un árbol significa visitar y procesar cada uno de sus nodos una sola vez siguiendo un orden determinado.

A diferencia de estructuras lineales como las listas, en los árboles no existe un “siguiente elemento” único, por lo que el orden de recorrido es muy importante.

La elección del recorrido depende de la aplicación:

- búsquedas,
- organización de datos,
- evaluación de expresiones,
- representación jerárquica, etc.

Existen dos enfoques principales de recorrido:

- **Recorrido en profundidad**
- **Recorrido en anchura**

Tipos de recorrido

Recorrido en profundidad

Se recorre un camino desde la raíz hasta el descendiente más lejano antes de volver atrás y continuar por otro camino.

Idea clave: Primero se visitan todos los descendientes de un nodo antes de pasar al siguiente.

Recorrido en anchura

El árbol se recorre nivel por nivel, avanzando horizontalmente.

Idea clave: Se procesa completamente un nivel antes de pasar al siguiente.

Secuencia de Recorridos

Los árboles binarios no almacenan sus elementos de forma lineal como ocurre en un arreglo o una lista enlazada. Debido a su estructura jerárquica, para acceder, visualizar o procesar la información es necesario recorrer sus nodos siguiendo un determinado orden.

A este proceso se lo denomina **recorrido del árbol** y consiste en visitar sistemáticamente cada nodo exactamente una vez. En los árboles binarios, donde cada nodo puede tener un subárbol izquierdo y un subárbol derecho, existen distintas formas de realizar este recorrido en profundidad, dependiendo del orden en que se visite la raíz y sus subárboles.

Secuencia de Recorrido

Tradicionalmente, se utiliza la siguiente notación:

- **N** → Nodo raíz
- **I** → Subárbol izquierdo
- **D** → Subárbol derecho

A partir de esta convención surgen los tres recorridos clásicos de árboles binarios:

- **Preorden (N-I-D)**
- **Inorden (I-N-D)**
- **Postorden (I-D-N)**

Cada uno de estos recorridos tiene aplicaciones particulares y permite procesar la información del árbol de distintas maneras.

Recorrido Preorden (N-I-D)

El recorrido **preorden** procesa primero la raíz del árbol y luego continúa por los subárboles izquierdo y derecho.
Su lógica de recorrido prioriza el nodo principal antes que sus descendientes.

N → I → D

Secuencia:

1. Visitar la raíz
2. Recorrer el subárbol izquierdo
3. Recorrer el subárbol derecho

Idea clave: En preorden, cada nodo se procesa antes que sus hijos.

DOM en páginas web

Las páginas web se organizan internamente como árboles llamados **DOM (Document Object Model)**.

```
<html>
  <body>
    <div>
      <p>Hola</p>
    </div>
  </body>
</html>
```

Cuando el navegador construye la página, primero interpreta el elemento contenedor (`div`) y luego procesa el contenido interno (`p`).
Ese comportamiento sigue la lógica del recorrido preorden: primero el nodo padre y después sus hijos.

Recorrido En Orden (I-N-D)

El recorrido **en orden** procesa primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.
La raíz queda “en medio” de ambos subárboles.

I → N → D

Secuencia

1. Recorrer subárbol izquierdo
2. Visitar la raíz
3. Recorrer subárbol derecho

Idea clave: En inorder, la raíz se procesa entre los dos subárboles.

Índices y datos ordenados

El recorrido en orden es muy utilizado en estructuras de búsqueda porque permite obtener los datos ya ordenados automáticamente.

El recorrido visita los valores de menor a mayor sin necesidad de ordenar después.
Por eso esta técnica se utiliza en:

- índices de bases de datos,
- árboles binarios de búsqueda (ABB),
- motores de búsqueda y sistemas de consulta.

Recorrido Postorden (I-D-N)

El recorrido **postorden** procesa primero los subárboles y deja la raíz para el final. Su nombre proviene de *post*, que significa *“después”*.

I → D → N

Secuencia

1. Recorrer subárbol izquierdo
2. Recorrer subárbol derecho
3. Visitar la raíz

Idea clave: En postorden, la raíz se procesa después de sus hijos.

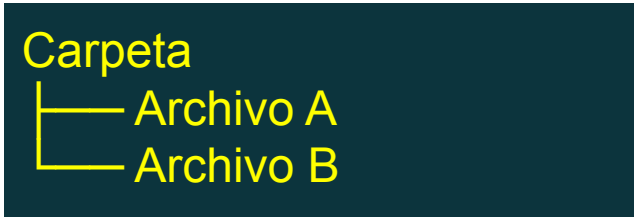
Eliminación de carpetas

El recorrido postorden se utiliza cuando primero deben procesarse los elementos internos antes de actuar sobre el contenedor principal.

Para eliminar correctamente la carpeta:

1. primero se eliminan los archivos,
2. y recién después la carpeta principal.

Ese comportamiento sigue la lógica: I → D → N



Repaso de Propiedades estructurales

Árbol lleno — cada nodo tiene 0 o 2 hijos, nunca 1 solo. No importa la altura ni el balance.

Árbol completo — todos los niveles están llenos excepto posiblemente el último, y en el último los nodos están todos a la izquierda.

Árbol perfectamente completo — todos los niveles están completamente llenos, sin excepción. Todos los nodos hoja están a la misma altura.

Árbol equilibrado — la diferencia de altura entre el subárbol izquierdo y derecho de cualquier nodo no supera 1.

Relación entre Arboles

- Perfectamente completo → es siempre lleno, completo y equilibrado. Es el caso más restrictivo.
- Completo → es casi siempre equilibrado, pero no necesariamente lleno.
- Lleno → no implica equilibrado ni completo.
- Equilibrado → no implica lleno ni completo.

La jerarquía sería así de restrictiva:

Perfectamente completo ⊂ Completo ⊂ Equilibrado

Y lleno es una condición independiente que puede combinarse o no con las anteriores.

Árbol de búsqueda binaria

Hasta ahora vimos árboles binarios sin ningún **orden** definido. ¿Qué pasa si le agregamos una regla?

Un **Árbol Binario de Búsqueda (ABB)** es un árbol binario donde se cumple siempre la siguiente propiedad:

- Todo nodo del subárbol **izquierdo** es **menor** que el nodo actual
- Todo nodo del subárbol **derecho** es **mayor** que el nodo actual

Esta regla se cumple para **cada nodo** del árbol, no solo para la raíz.

Árbol de búsqueda binaria

Un ABB es exclusivamente una **propiedad de orden**. No tiene ninguna condición de forma obligatoria.

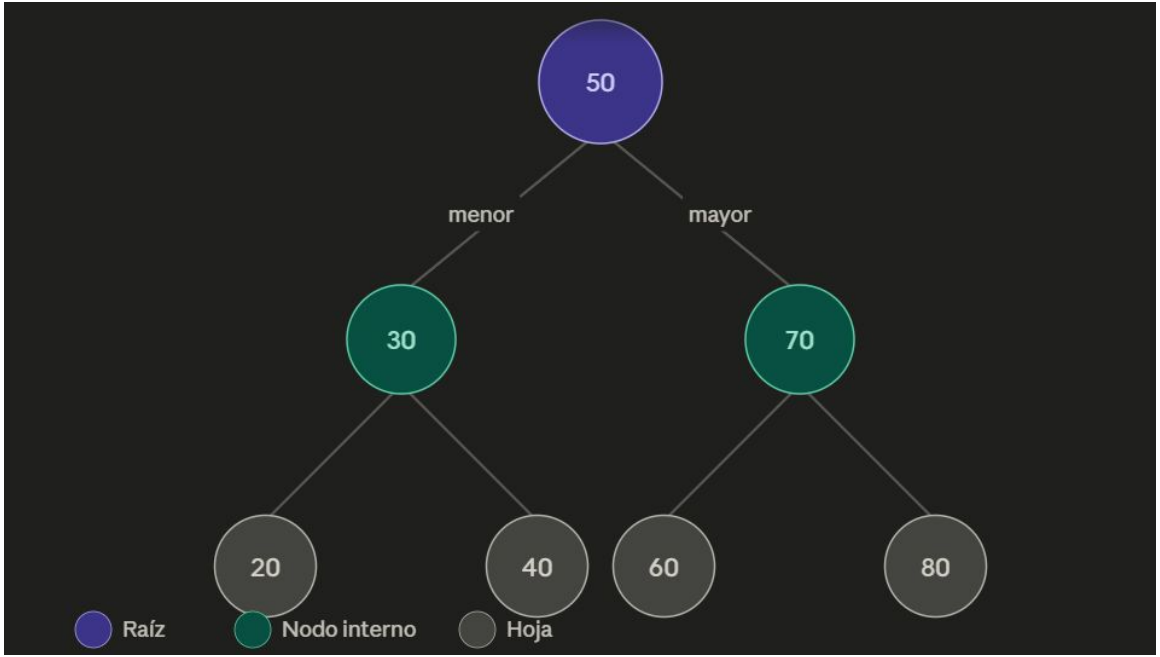
O sea un ABB puede ser:

- Perfectamente completo o no
- Equilibrado o no
- Lleno o no
- Incluso puede degenerarse en una lista

La **única condición** para ser ABB es la regla de orden: todo lo de la izquierda menor, todo lo de la derecha mayor. Nada más.

Cómo se ve un ABB?

Insertamos los valores: 50 · 30 · 70 · 20 · 40 · 60 · 80



Buscar en un ABB - *(BST Binary Search Tree)*

Buscar el valor **60** en el árbol anterior:

1. Comparamos con la raíz → 60 < 50? No → vamos a la **derecha**
2. Comparamos con 70 → 60 < 70? Sí → vamos a la **izquierda**
3. Comparamos con 60 → ¡encontrado! ✓

En cada paso **descartamos la mitad del árbol**. Por eso se llama de búsqueda — funciona igual que la búsqueda binaria pero sobre una estructura de árbol.

Eficiencia: en un árbol balanceado, buscar un elemento cuesta **O(log n)** comparaciones.

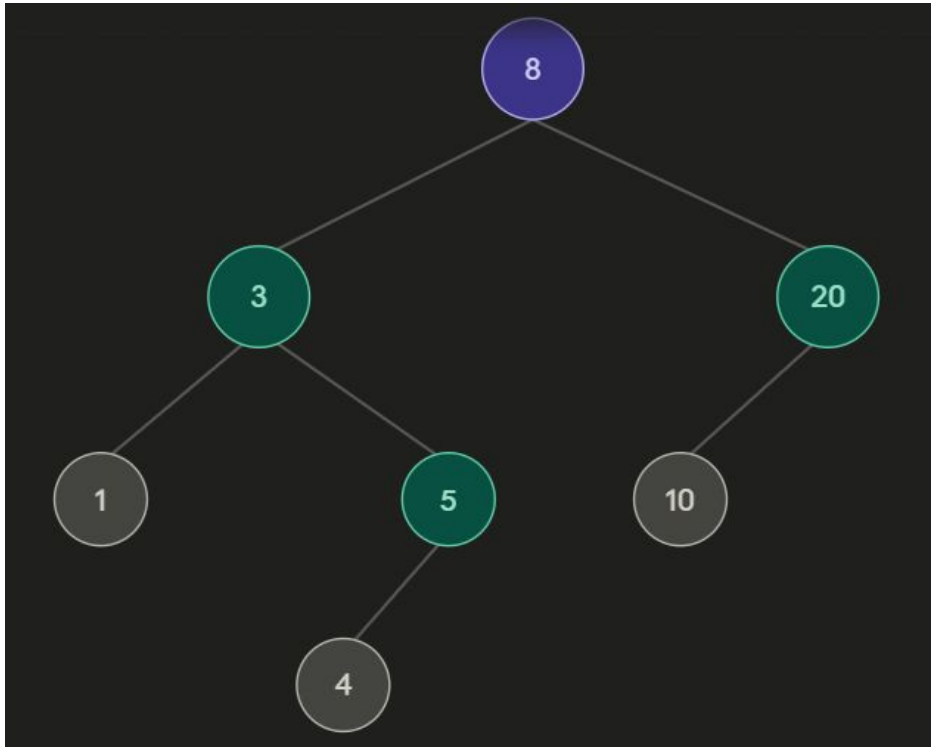
Cómo se crea un ABB

Inserción en un ABB

El árbol comienza vacío. Cada elemento nuevo se inserta siempre como una **hoja**, siguiendo esta regla en cada nodo que se visita:

- Si el valor nuevo es **menor** → ir a la izquierda
- Si el valor nuevo es **mayor** → ir a la derecha
- Repetir hasta encontrar un lugar vacío

Ejemplo con: 8 → 3 → 1 → 20 → 10 → 5 → 4



Los ABB no son únicos

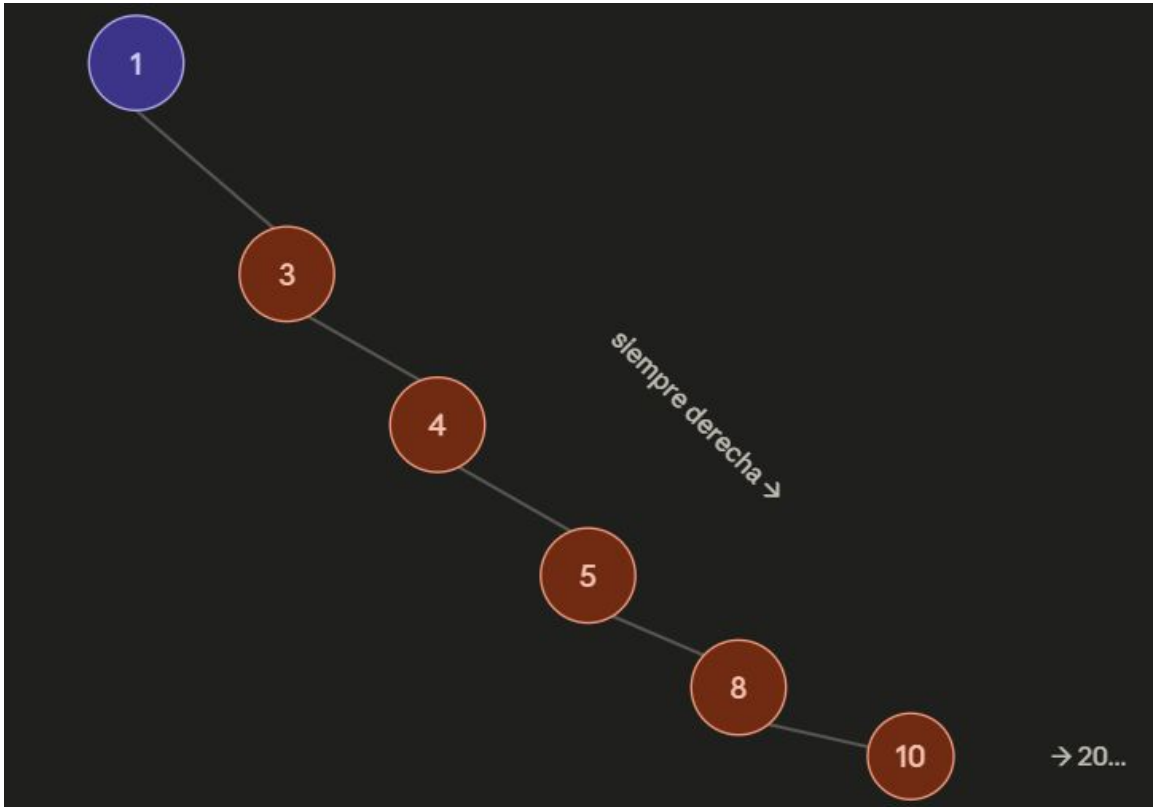
El árbol resultante **depende del orden de inserción**. Los mismos datos insertados en distinto orden producen árboles con distinta forma.

Con los mismos valores 8, 3, 1, 20, 10, 5, 4 pero insertados en otro orden, el árbol final sería diferente — aunque sigue siendo un ABB válido.

Esto tiene consecuencias en la eficiencia de búsqueda: un árbol desbalanceado puede degradarse hasta comportarse como una lista.

Los ABB no son únicos

Mismos datos: 1, 3, 4, 5, 8, 10, 20 — pero insertados **en orden creciente**. Cada nodo nuevo siempre es mayor que el anterior, entonces siempre va a la derecha.



Los ABB no son únicos

La diferencia es clara. El primer árbol tiene altura 4 y está relativamente balanceado — buscar cualquier elemento toma pocos pasos. El segundo se convirtió en una lista disfrazada de árbol, y buscar el último elemento requiere recorrerlo completo.

Eso es exactamente lo que dice la teoría: **los ABB no son únicos para los mismos datos**, y el orden de inserción determina tanto la forma como la eficiencia.

Operaciones en Árboles ABB

Los ABB tienen naturaleza recursiva, por lo que sus operaciones pueden implementarse de forma recursiva o iterativa.

Búsqueda — recorre el árbol comparando valores hasta encontrar el nodo buscado o confirmar que no existe. Devuelve la referencia al nodo o null.

Inserción — crea un nuevo nodo con su dato y lo ubica en la posición correcta respetando el orden del árbol.

Borrado — elimina el nodo que contiene el dato buscado. El árbol resultante debe seguir siendo un ABB válido.

Recorrido — visita todos los nodos del árbol en un orden determinado: preorden, enorden o posorden.

Búsqueda

La búsqueda de un nodo comienza en el nodo raíz y sigue estos pasos:

1. La clave buscada se compara con la clave del nodo raíz.

2. Si las claves son iguales, la búsqueda se detiene.

3. Si la clave buscada es mayor que la clave raíz, la búsqueda se reanuda en el subárbol

derecho. Si la clave buscada es menor que la clave raíz, la búsqueda se reanuda con el subárbol izquierdo.

Inserción en un ABB

Para insertar un nodo se sigue el mismo camino que haría una búsqueda. El nuevo nodo se coloca exactamente donde la búsqueda termina, por lo tanto **siempre se inserta como hoja**. El árbol resultante sigue siendo un ABB válido.

El algoritmo tiene dos casos:

Si el elemento ya existe → no se hace nada.

Si el elemento no existe → se inserta en el lugar exacto donde habría estado si existiera, es decir donde la búsqueda terminó sin encontrarlo.

Eliminación en un ABB

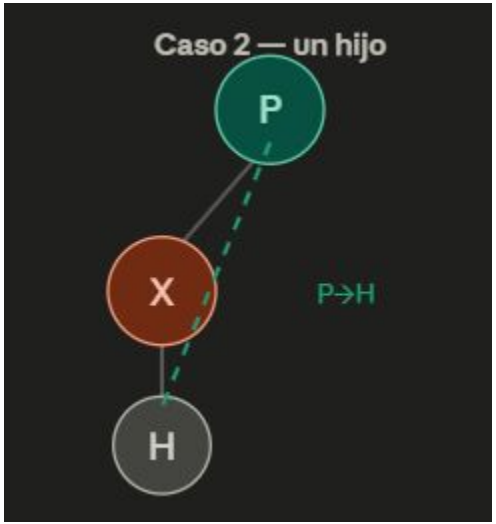
Buscar el nodo es el primer paso. Luego se presentan tres casos según la cantidad de hijos:

Nodo hoja — sin hijos. Se elimina directamente y el padre apunta a null.



Eliminación en un ABB

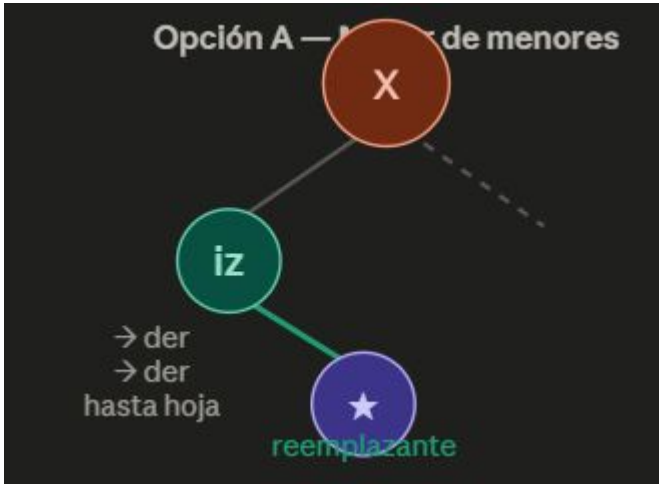
Un solo hijo — el padre del nodo eliminado apunta directamente al hijo. El nodo queda fuera de la cadena.



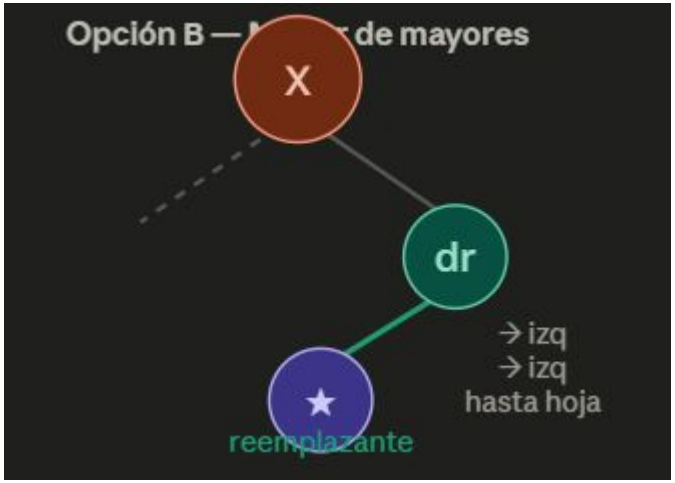
Eliminación en un ABB

Dos hijos — no se puede eliminar directamente. Hay que reemplazar su valor por el nodo más próximo que mantenga la propiedad ABB.

⚠ El caso 3 es el más complejo. El árbol resultante siempre debe seguir siendo un ABB válido.



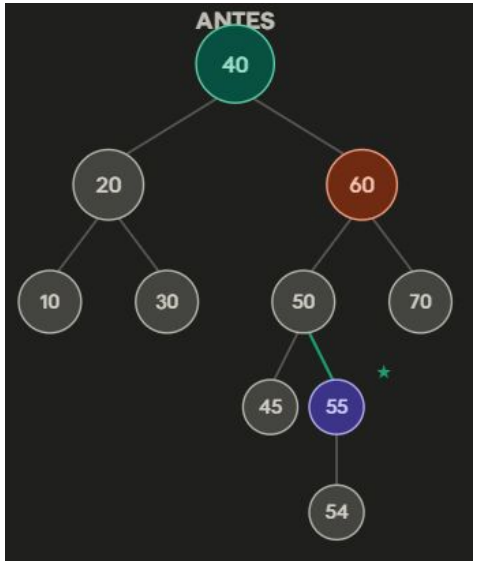


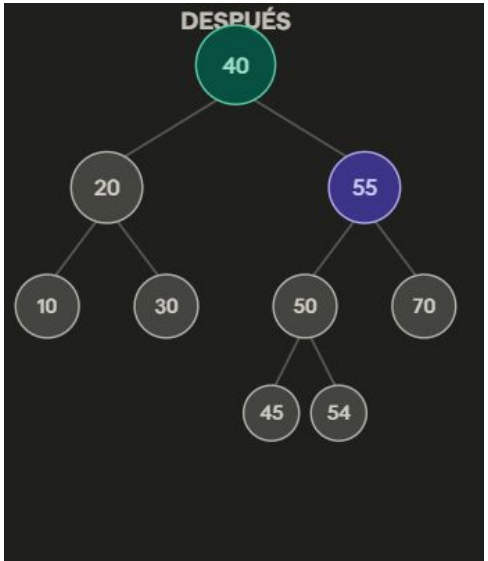


Ejemplo

Mayor de los menores — bajás por la rama **izquierda** del nodo a borrar y de ahí siempre a la **derecha** hasta no poder más. Ese nodo es el reemplazante.

Menor de los mayores — bajás por la rama **derecha** del nodo a borrar y de ahí siempre a la **izquierda** hasta no poder más.





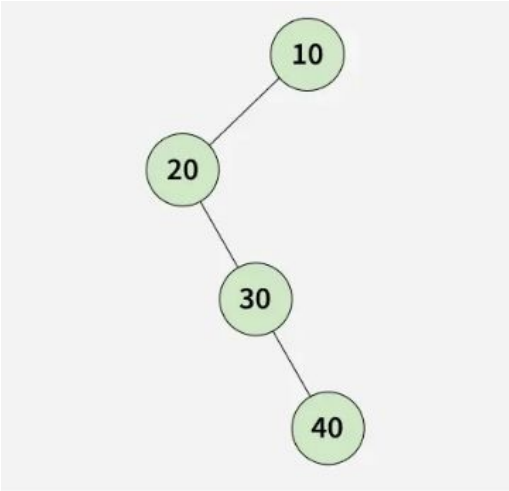
Es Importante el Forma?

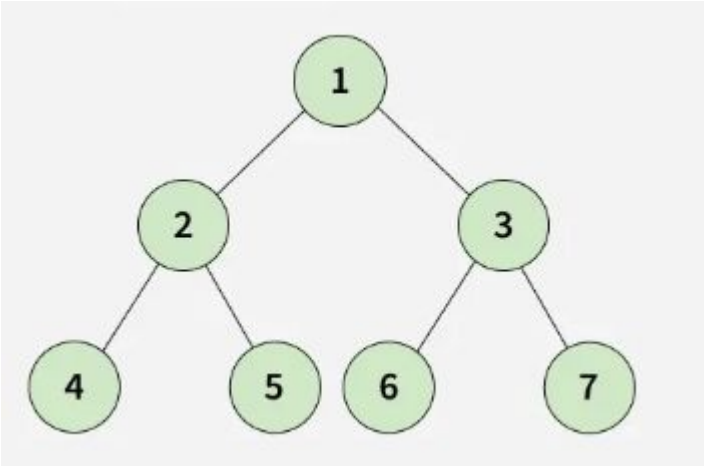
Ahora, en la práctica esto importa porque un ABB sin ninguna condición de forma adicional puede volverse muy ineficiente. Por eso existen variantes que combinan la propiedad de orden del ABB con propiedades de forma, como los **árboles AVL** que son ABB + equilibrado, o los **árboles Rojo-Negro** que también garantizan balance.

Limitaciones de los ABB

Los árboles binarios de búsqueda funcionan correctamente cuando los datos mantienen cierta organización jerárquica.

Cuando las inserciones y eliminaciones ocurren sin un orden predecible, el árbol puede desbalancearse y disminuir su eficiencia.





Árboles AVL

Cómo mantener eficiente un árbol aun cuando los datos se insertan sin orden?

Los árboles AVL son árboles binarios equilibrados diseñados para conservar una estructura balanceada y optimizar las operaciones de búsqueda, inserción y eliminación.

Origen histórico: desarrollados en **1962** por Adelson-Velsky y Landis.

Mantener el árbol balanceado automáticamente después de inserciones y eliminaciones, evitando pérdidas importantes de eficiencia.

Relación entre estructura y eficiencia de búsqueda

La eficiencia de un árbol binario de búsqueda no depende únicamente de la cantidad de nodos, sino de **cómo se distribuyen dentro del árbol**.

Cuando los elementos se insertan en **orden creciente o decreciente**, el árbol pierde balance y se comporta de forma secuencial.

$$O(n)$$

Cuando las claves menores y mayores se distribuyen de forma relativamente uniforme, el árbol conserva una estructura balanceada.

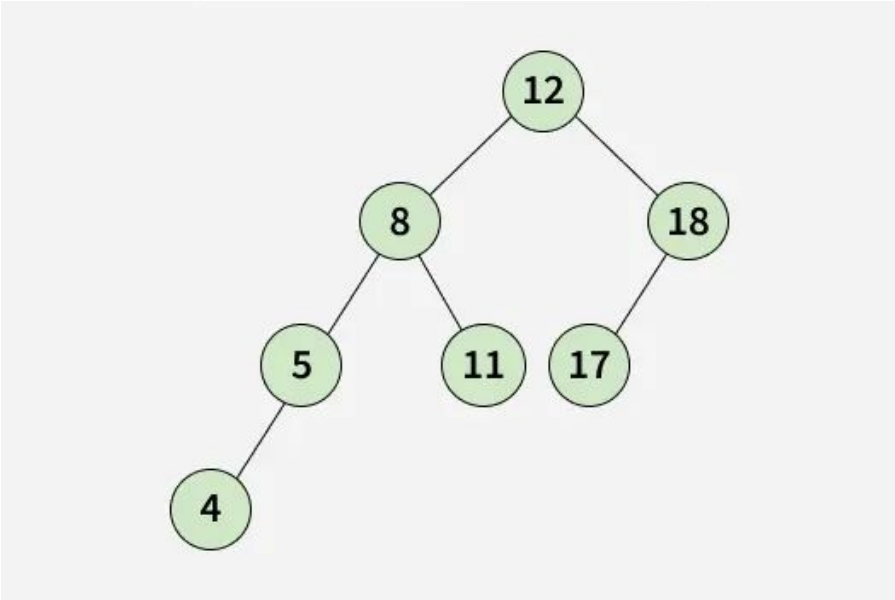
$$O(\log n)$$

La forma del árbol determina directamente el costo de búsqueda, inserción y eliminación. Para conservar eficiencia surgen los **árboles casi equilibrados (AVL)**.

Árbol AVL

Un árbol AVL es un árbol de búsqueda binaria (BST) autoequilibrado donde la diferencia entre las alturas de los subárboles izquierdo y derecho de todos los nodos es como máximo 1.

El árbol que se muestra es AVL porque la diferencia entre las alturas de los subárboles izquierdo y derecho de cada nodo es ≤ 1.

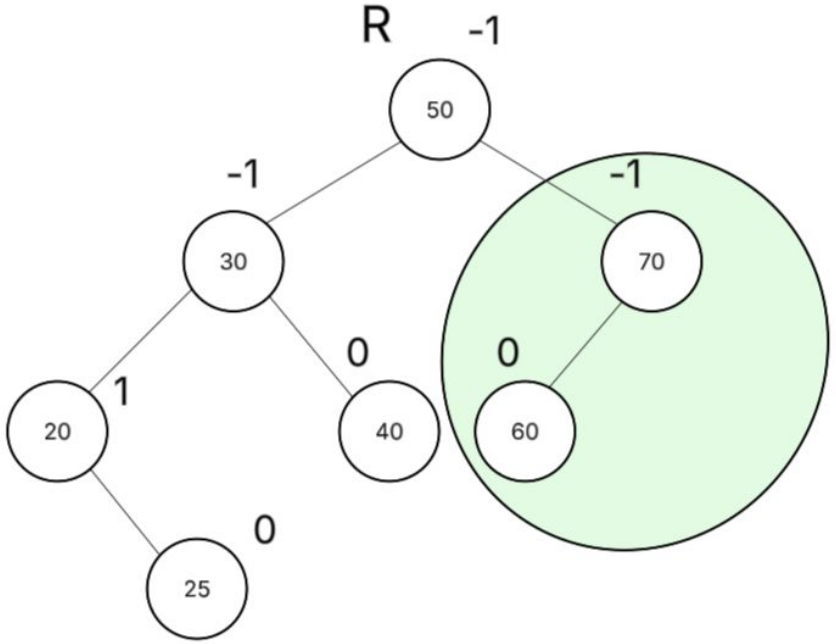


Factor de equilibrio en un árbol AVL

Un árbol AVL se considera equilibrado cuando la diferencia de alturas entre los subárboles izquierdo y derecho de cada nodo es mínima.

Definición del factor de equilibrio:

$FE = h_{derecho} - h_{izquierdo}$



Altura de un AVL

La altura de un árbol AVL es un parámetro determinante, ya que representa la cantidad máxima de niveles que deben recorrerse desde la raíz hasta una hoja. En consecuencia, el rendimiento de las operaciones de búsqueda, inserción y eliminación depende directamente de ella

Para estudiar su comportamiento, se analiza el **peor caso posible**: un árbol AVL con la **máxima altura permitida para una cantidad dada de nodos**. Bajo esta condición, el árbol conserva el equilibrio mínimo aceptable, por lo que sus subárboles difieren solo en una unidad de altura

Esta construcción genera una relación recursiva

$$N_h = N_{h-1} + N_{h-2} + 1$$

$$h \approx 1{,}44 \cdot \log(n)$$

Altura de un AVL con muchos datos

Supongamos un árbol AVL con:

$$n = 100000 \text{ nodos}$$

Para estimar la altura máxima en el peor caso se utiliza:

$$h \approx 1.44 \log(n)$$

Reemplazando:

$$h \approx 1.44 \log(100000) \qquad h \approx 24$$

Aun almacenando **100 000 elementos**, el árbol necesita recorrer aproximadamente **24 niveles** en el peor caso para encontrar un dato.

El equilibrio del árbol reduce drásticamente la cantidad de comparaciones necesarias.

Inserción en árboles AVL

La inserción en un árbol AVL sigue el mismo procedimiento que en un **árbol binario de búsqueda**: la nueva clave se compara desde la raíz y desciende por la rama izquierda o derecha hasta ubicarse como **nodo hoja**.

Sin embargo, la nueva inserción puede modificar la altura de una rama y alterar el **factor de equilibrio** de algunos nodos.

Por este motivo, el algoritmo no finaliza al insertar el dato: debe **volver por el camino recorrido**, verificando si el árbol continúa balanceado.

> Cuando el equilibrio se pierde, el árbol AVL aplica **rotaciones** para recuperar su estructura balanceada.

Actualización del factor de equilibrio

La inserción en un árbol AVL comienza igual que en un árbol binario de búsqueda: el nuevo elemento desciende hasta encontrar su posición y se agrega como **nodo hoja**, con factor de equilibrio igual a **0**.

Sin embargo, el proceso no termina allí. El algoritmo debe recorrer el camino en sentido inverso, desde el nuevo nodo hacia la raíz, actualizando el **factor de equilibrio (fe)** de los nodos afectados.

Solo los nodos que forman parte del **camino de búsqueda** pueden modificar su equilibrio.

Hay Casos en que las inserciones mejoran el balance!

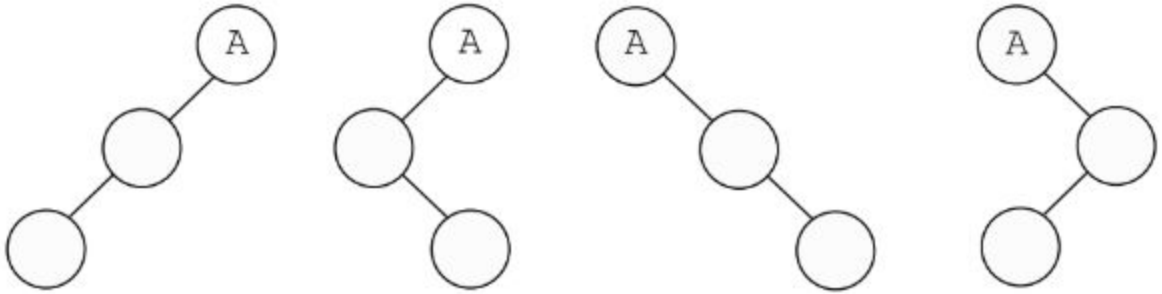
Pérdida del equilibrio y necesidad de rotación

Cuando un nodo ya presenta una diferencia de altura de **1**, una nueva inserción en la rama más alta puede incrementar la diferencia a **2**, rompiendo la condición AVL.

En ese momento el árbol debe **reestructurarse** mediante rotaciones.

Las situaciones posibles dependen del lugar donde ocurrió la inserción:

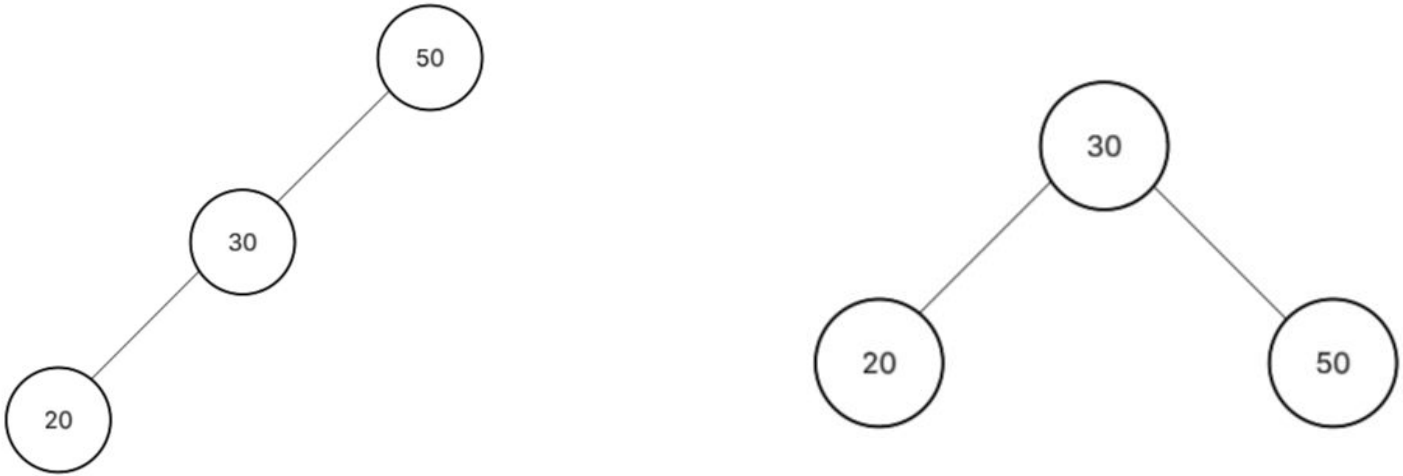
1. Izquierda – Izquierda (LL)
2. Izquierda – Derecha (LR)
3. Derecha – Derecha (RR)
4. Derecha – Izquierda (RL)



Aplicación de rotaciones para recuperar el equilibrio

Rotación simple (LL)

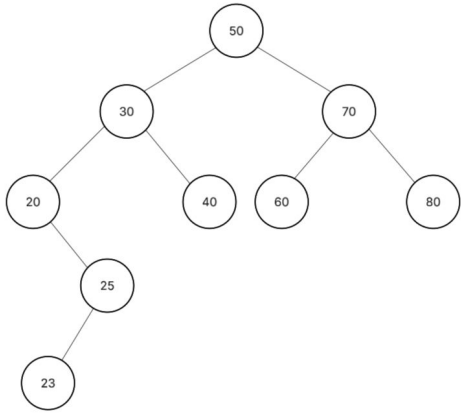
La inserción ocurre en la misma dirección (**izquierda–izquierda**), por lo que una sola rotación corrige el desbalance.

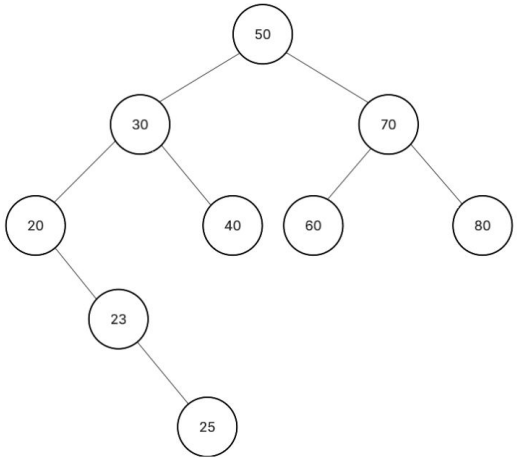


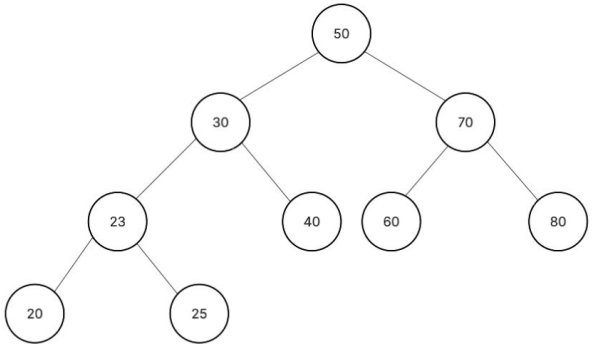
Aplicación de rotaciones para recuperar el equilibrio

Rotación doble (LR)

La inserción genera un crecimiento en forma de **zig-zag**, por lo que es necesario realizar dos rotaciones.







Borrado en un árbol AVL

El borrado en un árbol AVL comienza igual que en un **árbol binario de búsqueda**: primero se localiza el nodo y luego se elimina según el caso correspondiente.

Caso 1
Nodo hoja o con un hijo
→ Se elimina o reemplaza directamente

Caso 2
Nodo con dos hijos
→ Se reemplaza por el mayor del subárbol izquierdo (predecesor inorden)

La eliminación puede reducir la altura de una rama, por lo que el árbol debe regresar hacia la raíz actualizando el **factor de equilibrio (FE)**.

En AVL, borrar un nodo puede modificar la altura del árbol y romper el equilibrio.

Borrado en un árbol AVL

Durante la inserción, al realizar una rotación el proceso generalmente finaliza, porque la altura del subárbol vuelve a su estado anterior.

En cambio, al eliminar un nodo puede ocurrir que la altura del subárbol **continúe disminuyendo**, afectando otros nodos superiores.

Por esta razón, el algoritmo debe seguir revisando el camino hacia la raíz.

Inserción
Insertar → revisar → rotar → FIN

Borrado
Eliminar → revisar → rotar
↓
seguir revisando

Un borrado en AVL puede requerir **más de una rotación**, aunque su complejidad total sigue siendo:

O(logn)